

TECHNICAL SPECIFICATION

NutriAI

Intelligent AI Diet Recommendation Platform
Software Architecture & Technical Specification

VERSION	TYPE	DATE	CLASSIFICATION
1.0.0	Full Stack Spec	July 2026	Internal

SYSTEM ARCHITECTURE OVERVIEW

React Native	Express.js	PostgreSQL
Vision AI	Meal AI	Redis
Firebase	AWS S3	

1. Executive Summary

NutriAI is an AI-powered diet recommendation and food tracking platform designed to compete directly with industry leaders such as Healthify, MyFitnessPal, Lose It!, and Noom. The platform differentiates itself through a vision-first approach: instead of requiring users to manually search for foods or painstakingly count calories, NutriAI allows users to simply photograph their meals. A sophisticated Vision AI model recognizes the food items, estimates serving sizes, and matches them against a curated database of over 5,000 meals. The system then automatically calculates nutritional values and updates the user's daily progress in real time.

Unlike competitors that rely on crowdsourced databases and manual entry, NutriAI integrates only two AI models into its architecture. The first is a Vision AI model (powered by GPT-4.1 Vision, GPT-5, or Claude Vision) responsible solely for food recognition from images and text descriptions. The second is a lightweight Meal Recommendation AI (Qwen 2.5 3B or Gemma 3 4B) that selects the best three meal options from a curated candidate list based on the user's nutritional goals, preferences, and consumption history. All other logic, including BMI calculation, BMR/TDEE computation, macro distribution, meal filtering, and scoring, is handled by deterministic backend algorithms. This design minimizes AI dependency, reduces latency, and cuts infrastructure costs significantly.

The technology stack is deliberately modern and performant. The mobile application is built with React Native, enabling a single codebase for both iOS and Android platforms. The backend runs on Express.js with Node.js, chosen for its event-driven architecture and excellent JSON handling capabilities. PostgreSQL serves as the primary relational database, storing user profiles, meal data, food logs, and progress tracking records. Redis is employed as a caching layer for session management, frequently accessed meal data, and rate limiting. AWS S3 handles image storage for food photographs, while Firebase Cloud Messaging (FCM) delivers push notifications for meal reminders and goal

celebrations.

The platform targets health-conscious individuals aged 18 to 55 across multiple geographic markets, with initial focus on India, the Middle East, Southeast Asia, and North America. Revenue will be generated through a freemium model: a free tier with basic food logging and ad-supported content, and a premium tier at \$9.99/month offering personalized AI meal plans, detailed analytics, unlimited photo recognition, and integration with wearable devices. NutriAI's competitive advantage lies in its simplicity, accuracy, and speed. By reducing the friction of food logging from minutes per meal to seconds per photo, the platform aims to achieve significantly higher daily active usage rates than manual-entry competitors.

2. Product Vision

Our vision is to build the most accurate and effortless AI diet tracker that works simply by photographing food. We believe the future of nutrition tracking should not involve manual searching through thousands of food items, tedious calorie counting, or complex data entry. The application should feel instant, clean, and minimal, drawing design inspiration from Healthify's polished interface while incorporating cutting-edge AI capabilities that set it apart from existing solutions. The core philosophy is: point, shoot, done. Within two seconds of taking a photo, the user should see their meal identified, nutrition calculated, and daily progress updated.

The product experience centers on three pillars: **Accuracy**, **Speed**, and **Simplicity**. Accuracy means the Vision AI must correctly identify cooked meals across diverse cuisines, estimate realistic serving sizes, and match foods to precise nutritional profiles in our database. Speed means the entire recognition pipeline, from image capture to result display, must complete within three seconds on a standard 4G connection. Simplicity means the user interface should require zero training. A new user should be able to open the app, point their camera at their lunch, and have their meal logged without reading a single

tutorial.

The long-term product roadmap extends beyond basic tracking. We envision NutriAI becoming a comprehensive nutritional intelligence platform that learns from millions of user interactions to improve food recognition accuracy, refines meal recommendations based on anonymized aggregate outcomes, and eventually offers predictive insights such as "users with similar profiles who ate this combination achieved their weight loss goals 23% faster." Integration with wearable devices (Apple Watch, Fitbit, Garmin) will enable automatic activity tracking that feeds directly into the TDEE calculation, making the system progressively more accurate without any additional user effort. The ultimate goal is to make NutriAI the default tool that health professionals recommend to their patients for dietary management.

3. Business Flow

The overall business flow of NutriAI follows a carefully designed user lifecycle from initial download through daily engagement. Each stage has been optimized to minimize friction and maximize user retention. The complete workflow proceeds through the following stages:

Download App → Create Account / Auth → Onboarding Questionnaire →
Backend: Calculate BMI, BMR, TDEE, Target Calories, Macros →
AI Generates Day 1 Meal Plan → Dashboard with Meal Cards →
User Logs Food (3 methods: "I Ate This", Photo Upload, Text Desc) →
Food Recognition AI → Database Lookup → Nutrition Calculation →
Update Remaining Calories → Recommend Next Meals →
End-of-Day Summary → Auto-Plan Tomorrow → Repeat Daily

3.1 Onboarding Stage

During onboarding, the user completes a comprehensive questionnaire that collects age, gender, height, weight, fitness goal, lifestyle activity level, workout frequency, food cuisine preference, dietary restriction, allergy profile, and monthly food budget. This data is processed by the Nutrition Engine to

calculate BMI (Body Mass Index), BMR (Basal Metabolic Rate) using the Mifflin-St Jeor equation, TDEE (Total Daily Energy Expenditure) by applying an activity multiplier, target daily calories adjusted by the selected goal (with a safe +/- 500 kcal delta), and optimal macronutrient split ratios. The entire calculation happens on the backend and takes less than 100 milliseconds.

3.2 Daily Engagement Loop

Each morning, the user opens the app to find a pre-generated dashboard displaying today's meal plan across four meal slots: Breakfast, Lunch, Dinner, and Snack. Each meal slot contains three carefully curated meal options, generated by the Recommendation Engine filtering the 5,000-meal database through a multi-stage pipeline (allergy removal, cuisine matching, calorie budget enforcement, protein minimum enforcement, variety scoring) and then ranked by the Meal Recommendation AI. The user selects "I Ate This" for one of the options, uploads a photo of their actual meal, or types a description. The system immediately calculates consumed nutrition, updates the remaining budget, and recalculates recommendations for the remaining meal slots to ensure the user stays on track throughout the day.

3.3 End-of-Day and Auto-Planning

At the end of each day, NutriAI generates a comprehensive summary showing total calories consumed, macro achievement percentages, water intake, and a streak counter for consecutive days on target. This summary is delivered via push notification and is also available in the Progress tab. Based on the day's consumption patterns, the system automatically generates the next day's meal plan during off-peak hours, ensuring the dashboard is ready when the user wakes up. The auto-planning process factors in any residual calorie carryover from the previous day and adjusts the next day's targets accordingly.

4. User Journey

The first-time user experience is critical for retention. NutriAI guides new users through a seamless onboarding journey that collects essential data while maintaining engagement. Below is the step-by-step walkthrough of a new user's experience from app launch to their first logged meal:

4.1 Step 1: App Launch and Registration

The user downloads NutriAI from the App Store or Google Play and launches the application. The splash screen displays for 1.5 seconds with a branded animation. The user is presented with a clean registration screen offering email/password signup, Google OAuth, and Apple Sign In. After selecting their preferred method, the account is created and a welcome notification is sent. The entire registration process takes under 30 seconds.

4.2 Step 2: Personal Profile Collection

The user enters their basic information: first name, age, gender (Male/Female/Other), height (in cm or feet/inches), weight (in kg or lbs), country of residence, and timezone. This data is essential for the Nutrition Engine to perform accurate BMI and BMR calculations. The interface uses large, tappable input fields with intelligent defaults based on the device locale.

Field	Type	Options / Description	Required
Name	Text	First and last name	Required
Age	Number	18-100	Required

Field	Type	Options / Description	Required
Gender	Select	Male / Female / Other	Required
Height	Number + Unit	cm or ft/in	Required
Weight	Number + Unit	kg or lbs	Required
Country	Search Select	ISO country list	Required
Timezone	Auto-detect	Device timezone	Auto

4.3 Step 3: Goal and Lifestyle Selection

The user selects their primary fitness goal from six options: Muscle Gain (caloric surplus, high protein), Lose Fat (caloric deficit, high protein), Maintain Weight (maintenance calories), Body Recomposition (slight deficit, very high protein), Weight Gain (significant surplus), and Athlete (performance-focused macros). Next, the user selects their lifestyle activity level: Office Worker (sedentary), Student (light activity), Construction Worker (moderate), Athlete (high activity), or Sedentary (minimal movement). The workout frequency is then chosen from: None, 1-2 days per week, 3-5 days per week, 6 days per week, or Twice Daily.

Goal	Calorie Strategy	Macro Split
Muscle Gain	+500 kcal surplus	35% protein, 40% carbs, 25% fat
Lose Fat	-500 kcal deficit	35% protein, 35% carbs, 30% fat
Maintain	TDEE maintenance	30% protein, 40% carbs, 30% fat
Recomp	-250 kcal deficit	40% protein, 35% carbs, 25% fat
Weight Gain	+750 kcal surplus	25% protein, 45% carbs, 30% fat
Athlete	+300 kcal surplus	30% protein, 45% carbs, 25% fat

4.4 Step 4: Food Preferences and Allergies

The user selects their preferred cuisine from: Indian, Arabian, Chinese, Italian, or Mixed (any cuisine). The dietary restriction is chosen from: Vegetarian, Non-Vegetarian, Vegan, or Eggetarian. The allergy profile allows multiple selections from: Milk, Peanut, Seafood, Egg, Gluten, Soy, Tree Nuts, and Shellfish. Finally, the monthly food budget is selected from: Low (home-cooked, affordable ingredients), Medium (moderate variety), or Premium (gourmet and specialty options). This completes the onboarding questionnaire, and the backend immediately calculates the user's full nutritional profile.

4.5 Step 5: First Dashboard Experience

The backend calculates BMI, BMR, TDEE, target calories, and macro targets within milliseconds. The Recommendation Engine processes the 5,000-meal database through its filtering pipeline and generates the first day's meal plan with three options per meal slot. The user lands on the Dashboard, which displays a greeting message, today's calorie goal with a circular progress indicator, protein tracker bar, and four meal cards (Breakfast, Lunch, Dinner, Snack) each containing three meal options. The user taps "I Ate This" on their first meal, and the progress ring animates to reflect the updated nutrition. A celebratory micro-animation confirms the action, and the remaining meal recommendations are instantly recalculated.

5. Complete UI Wireframes

The NutriAI mobile interface follows a minimal Healthify-style design language: clean cards, generous whitespace, a neutral color palette with accent highlights, and large touch targets. Below are the wireframe specifications for each screen, presented in tabular format for precision.

5.1 Dashboard Screen

The Dashboard is the primary screen and the landing page after authentication. It presents the user's daily nutrition overview and meal plan in a single scrollable view.

Component	Description	Position
Header Bar	App logo (left) "Hi, [Name]" (center) Notification bell (right)	Fixed
Calorie Ring	Circular progress ring showing consumed/target calories. Center: "1,450 / 2,200 kcal"	Top card
Macro Bars	Three horizontal bars: Protein (green), Carbs (blue), Fat (orange) with grams and percentage	Below ring
Meal Cards (x4)	Breakfast / Lunch / Dinner / Snack cards. Each card shows 3 meal options with estimated calories and protein	Scrollable
Meal Option	Meal name 450 kcal 32g protein "I Ate This" button (green accent)	Inside card
Bottom Nav	Home (active) Log Camera Progress Profile	Fixed bottom

5.2 Food Log Screen

Component	Description	Position
Date Selector	Horizontal date strip showing last 7 days, today highlighted	Top
Today Summary	Total calories consumed, remaining calories, protein progress	Card
Logged Meals	List of today's logged meals with time, food name, calories, protein	Scroll list
Add Meal FAB	Floating "+" button that opens 3 options: Photo, Search, Quick Add	Bottom right
Photo Option	Opens camera with overlay guide frame for food	Modal
Search Option	Search bar with autocomplete from meal database	Modal

5.3 Camera Screen

Component	Description	Position
Camera Viewfinder	Full-screen camera preview with semi-transparent overlay	Full screen
Scan Frame	Centered rectangular frame with rounded corners, food should be centered	Center
Flash Toggle	Icon button to toggle flash on/off	Top right
Gallery Button	Option to select existing photo from gallery	Bottom left
Capture Button	Large circular shutter button	Bottom center
Status Text	"Point camera at your meal" or "Recognizing..." during processing	Top center

5.4 Progress Screen

Component	Description	Position
Period Selector	Tabs: Week Month 3 Months All Time	Top
Weight Chart	Line chart showing weight trend over selected period	Card

Component	Description	Position
Calorie Chart	Bar chart showing daily calorie intake vs target	Card
Streak Counter	Fire icon with "7 day streak!" text	Card
Macro Averages	Average macro distribution over the period as pie chart	Card
Insights	AI-generated text insights: "Your protein intake improved 15% this week"	Card

6. UX Flows

Below are the detailed user experience flows for the primary interaction patterns within the NutriAI application. Each flow is designed to minimize the number of taps required and provide clear feedback at every step.

6.1 Food Logging Flow (Three Methods)

Step	Action	Details
1	Method Selection	User taps "+" FAB on Dashboard or Food Log screen
2	Method A: Tap "I Ate This"	Select from recommended meal cards directly on Dashboard
3	Method A Confirmation	Serving size confirmation popup: "Small (1 serving)" or adjust
4	Method A Complete	Nutrition updated, progress ring animates, toast "Meal logged!"
5	Method B: Photo Upload	User taps Camera icon, captures or selects photo
6	Method B Processing	Image compressed, uploaded to S3, sent to Vision AI
7	Method B Result	Food identified with confidence score, shown for confirmation
8	Method B Confirm	User confirms or corrects the identification, serving size adjusted

Step	Action	Details
9	Method C: Text Search	User taps Search, types meal name or description
10	Method C Autocomplete	Autocomplete suggestions appear from 5,000-meal database
11	Method C Select	User selects meal, adjusts serving, confirms
12	All Methods	Food logged, remaining calories recalculated, next meal recommendations updated

6.2 Meal Card Selection Flow

Step	Description
1	User scrolls Dashboard to desired meal slot (e.g., Lunch)
2	Three meal option cards displayed with: name, calories, protein, thumbnail
3	User taps desired meal card to expand details (full nutrition breakdown)
4	"I Ate This" button highlighted in accent green color
5	Tap "I Ate This" triggers serving size selector: Small / Medium / Large / Custom
6	Nutrition calculated for selected serving size and added to daily totals
7	Meal card shows checkmark, Dashboard progress ring updates with animation
8	Remaining meal slots automatically recalculate recommendations

6.3 Daily Summary View

Step	Description
1	User taps Progress tab or receives end-of-day push notification
2	Summary card displays: Total calories (consumed vs target), each macro (g consumed vs g target)
3	Percentage achievement circles: Calories (green if within 10%, yellow if within 20%, red otherwise)

Step	Description
4	Meal log timeline: chronological list of all meals logged today with timestamps
5	Water intake tracker: glasses consumed vs target
6	Streak counter: consecutive days within calorie target range
7	"Plan Tomorrow" button if user wants to customize next day's meals manually

7. High-Level Architecture

NutriAI follows a clean, modular architecture with clear separation between the mobile client, backend API layer, data persistence layer, and external AI services. The system is designed for horizontal scalability, with each major component independently deployable and observable.

```

+-----+ +-----+
| | HTTPS | Express.js API |
| React Native | <====> | Server (Node.js) |
| Mobile App | JSON | Port 3000 / 3001 |
| (iOS + Android) | +---+-----+-----+---+
+-----+ | |
| |
+-----+ +---+---+ +---+-----+
| PostgreSQL | | Redis | | AWS S3 |
| (Primary) | | (Cache) | | (Images) |
+-----+ +-----+ +-----+
+-----+ +---+-----+
| Firebase | | External AI |
| FCM | | Services |
| (Push Notif) | | +-----+ |
+-----+ | | GPT/Claude | |
| | Vision AI | |
| +-----+ |
| +-----+ |
| | Qwen/Gemma | |
| | Meal Rec AI | |
| +-----+ |
+-----+

```

The mobile application communicates exclusively with the Express.js API server over HTTPS/REST. The API server serves as the orchestration layer, routing requests to the appropriate service, managing authentication, enforcing rate limits, and coordinating calls to external AI services and data stores. PostgreSQL is the single source of truth for all persistent data including users, meals, food logs, and progress snapshots. Redis serves as the caching and session management layer, storing JWT session data, caching frequently accessed meal records, and implementing rate limiting counters. AWS S3 provides scalable object storage for food images uploaded by users, with presigned URLs for secure direct uploads from the mobile client. Firebase Cloud Messaging handles push notification delivery for meal reminders, daily summaries, and achievement celebrations.

External AI services are accessed through dedicated gateway modules within the API server. The Vision AI Gateway handles communication with GPT-4.1 Vision, GPT-5, or Claude Vision for food recognition tasks. The Meal Recommendation Gateway interfaces with a locally hosted or cloud-hosted Qwen 2.5 3B or Gemma 3 4B model for meal ranking. Both AI gateways implement circuit breaker patterns, request timeouts, response caching, and fallback strategies to ensure system resilience.

8. Low-Level Architecture

The NutriAI backend is decomposed into thirteen specialized services, each with a single responsibility and clear interface contracts. This microservice-inspired modular monolith approach allows for future extraction into independent services without architectural refactoring.

Service	Responsibility	Dependencies
AuthService	User authentication, JWT token generation, OAuth integration, password hashing, session management	Redis, PostgreSQL

Service	Responsibility	Dependencies
UserService	User profile CRUD operations, avatar management, account settings, deletion workflows	PostgreSQL, S3
OnboardingService	Multi-step onboarding orchestration, data validation, Nutrition Engine trigger	NutritionEngineService, PostgreSQL
NutritionEngineService	BMI/BMR/TDEE calculation, macro target computation, calorie adjustment logic	PostgreSQL
MealPlanService	Daily meal plan generation, plan caching, auto-scheduling for next day	RecommendationService, PostgreSQL
FoodRecognitionService	Image preprocessing, Vision AI gateway orchestration, result validation	Vision AI, FoodDatabaseService
FoodDatabaseService	Meal search, alias matching, nutrition data retrieval, bulk data management	PostgreSQL, Redis
RecommendationService	Candidate generation, filtering pipeline, scoring, AI ranking orchestration	Meal Rec AI, FoodDatabaseService
MealLogService	Food log creation, item management, daily nutrition aggregation, undo operations	PostgreSQL, NutritionEngineService
ProgressService	Progress snapshots, streak calculation, trend analysis, chart data preparation	PostgreSQL
NotificationService	Push notification scheduling, FCM integration, notification preferences	Firebase FCM, PostgreSQL
AnalyticsService	Event tracking, usage analytics, funnel metrics, anonymized data aggregation	PostgreSQL, Redis
WaterLogService	Water intake tracking, reminder scheduling, daily target monitoring	PostgreSQL, NotificationService

Each service follows a layered internal architecture with a Controller layer (request handling and validation), a Service layer (business logic), and a Repository layer (data access). Cross-service communication within the monolith occurs through direct function calls with well-defined interfaces. External communication uses REST APIs with JSON payloads. All services share a common error handling middleware, logging utility, and configuration management module. The dependency injection pattern is employed throughout, with a centralized container managing service instantiation and

lifecycle.

9. AI Architecture Overview

A core design principle of NutriAI is the deliberate minimization of AI model usage. The platform requires only **two AI models**, each with a narrowly scoped responsibility. This approach dramatically reduces infrastructure costs (from potentially hundreds of dollars per month for complex multi-model pipelines to under \$50), lowers latency (AI calls are a small fraction of the total request time), improves reliability (fewer external service dependencies), and makes the system easier to debug and maintain.

9.1 AI-1: Vision AI (Food Recognition)

The Vision AI model is responsible exclusively for identifying cooked food items from images and text descriptions. Supported providers include GPT-4.1 Vision, GPT-5 Vision (when available), and Claude 3.5/4 Vision from Anthropic. The model receives either a food photograph or a text description and returns a structured JSON response containing the identified food name, estimated serving size, and a confidence score between 0 and 1. Critically, the Vision AI does NOT calculate nutritional values, does NOT recommend meals, and does NOT make dietary judgments. Its output is simply a food identifier that the backend uses to look up nutrition data from the PostgreSQL meal database.

9.2 AI-2: Meal Recommendation AI

The Meal Recommendation AI is a lightweight language model responsible for ranking pre-filtered meal candidates. Supported models include Qwen 2.5 3B Instruct (ideal for self-hosting on a single GPU) or Gemma 3 4B (Google's efficient open model). This model receives a filtered list of up to 20 candidate meals along with the user's nutritional goals, remaining calorie/protein budget, and 7-day consumption history. It returns a ranked list of the top 3 meals with fit scores. The model does NOT perform any nutrition calculation, food recognition, or database operations. It is purely a ranking function that considers user preferences and variety constraints to select the most appealing options from an already-qualified candidate pool.

9.3 Everything Else is Backend Logic

All remaining intelligence in the system is implemented as deterministic backend algorithms: BMI calculation, BMR computation (Mifflin-St Jeor), TDEE estimation, calorie target adjustment, macro distribution, meal candidate filtering (allergy removal, cuisine matching, calorie budget enforcement), scoring formulas, meal distribution across slots, water intake recommendations, streak calculations, and progress trend analysis. This design ensures that NutriAI's core functionality operates at database query speed, with AI calls reserved only for the two tasks where human-like perception (visual food recognition) and nuanced preference matching (meal ranking) genuinely benefit from machine intelligence.

10. Vision AI Design

10.1 Purpose and Scope

The Vision AI module is designed with a single, clearly defined purpose: identify cooked food items from user-uploaded images or text descriptions. It serves as

the bridge between a user's visual input and the structured food database. The module must handle diverse cuisines, varying lighting conditions, partial visibility of food items, and multi-item plates (e.g., a tray with rice, curry, and salad). The identification must return a normalized food name that can be matched against the meal_aliases table in PostgreSQL.

10.2 Processing Pipeline

```
Upload Image → Client-side Compression (max 2MB, quality 0.7, 1024px)
→ S3 Presigned URL Upload → Store in S3 Bucket
→ POST /api/food-recognize { imageUrl }
→ Backend: Validate image format (JPEG/PNG/HEIC)
→ Vision AI Gateway: Build prompt + image
→ GPT/Claude Vision API Call (timeout: 10s)
→ Parse JSON Response → Validate confidence > 0.7
→ Alias Matching: Normalize name → Query meal_aliases
→ DB Lookup: Fetch meal_id, nutrition data
→ Return: { food_id, name, serving, calories, protein, confidence }
```

10.3 Expected JSON Output

```
{
  "foods": [
    {
      "name": "chicken biryani",
      "serving_description": "1 plate (approx 300g)",
      "serving_weight_grams": 300,
      "confidence": 0.92
    }
  ]
}
```

10.4 Service Responsibilities

#	Responsibility
1	['Accept food image or text description input']
2	['Compress and validate image format and size']
3	['Construct system prompt with food recognition instructions']

#	Responsibility
4	['Call Vision AI provider (GPT/Claude) with prompt + image']
5	['Parse and validate JSON response structure']
6	['Validate confidence threshold (minimum 0.7)']
7	['Normalize identified food names for database matching']
8	['Match normalized names against meal_aliases table']
9	['Return structured food identification result']

10.5 Not Responsible For

- Calculating nutritional values (handled by NutritionEngineService)
- Recommending meals (handled by RecommendationService)
- Storing food logs (handled by MealLogService)
- User authentication or authorization
- Image storage management (handled by S3 integration)
- Push notification delivery
- Any dietary advice or health recommendations

11. Meal Recommendation AI Design

11.1 Purpose and Scope

The Meal Recommendation AI has one job: choose the best three meals from a pre-filtered candidate list of up to 20 meals. It receives the user's current

nutritional context (goal, remaining calories, remaining protein, remaining carbs, remaining fat), the list of candidate meals with their nutritional profiles, the user's 7-day consumption history, and stated preferences. It returns a ranked list of exactly three meals with a fit score for each, indicating how well the meal matches the user's current needs and preferences.

11.2 Input Parameters

Parameter	Type	Description
user_goal	String	muscle_gain, lose_fat, maintain, recomp, weight_gain, athlete
remaining_calories	Number	Calories remaining for the current meal slot
remaining_protein	Number	Grams of protein remaining for the day
remaining_carbs	Number	Grams of carbs remaining for the day
remaining_fat	Number	Grams of fat remaining for the day
candidates	Array	Up to 20 meals: { id, name, calories, protein, carbs, fat, cuisine, tags }
history_7day	Array	Meal IDs consumed in the last 7 days for variety scoring
preferences	Object	User cuisine preference, diet type, budget level

11.3 Output Format

```
{
  "recommendations": [
    { "meal_id": 1042, "name": "Grilled Chicken Salad", "fit_score": 0.94 },
    { "meal_id": 2087, "name": "Lentil Soup with Brown Rice", "fit_score": 0.87 },
    { "meal_id": 3415, "name": "Greek Yogurt Parfait", "fit_score": 0.81 }
  ]
}
```

11.4 Not Responsible For

- Nutrition calculation (BMI, BMR, TDEE, macro targets) - handled by NutritionEngineService

- Food recognition from images - handled by FoodRecognitionService
- Generating candidate meals - handled by RecommendationService filtering pipeline
- Database operations - handled by FoodDatabaseService
- User preference storage - handled by UserService

12. Prompt Engineering

12.1 Vision AI System Prompt

The Vision AI prompt is engineered to be extremely prescriptive, eliminating ambiguity and preventing the model from attempting tasks outside its scope. Every instruction is designed to constrain the output to exactly what the backend pipeline expects.

You are a food recognition assistant for a diet tracking application.

Your ONLY task is to identify cooked food items visible in the image or described in the text. Follow these rules strictly:

1. Recognize ONLY cooked/prepared foods. Ignore raw ingredients, packaged items with visible nutrition labels, and non-food objects.
2. Estimate the serving size in grams based on the visible portion.
3. Provide a confidence score (0.0 to 1.0) for each identification.
4. Return ONLY valid JSON. No markdown, no explanation, no commentary.
5. Do NOT calculate nutritional values (calories, protein, carbs, fat).
6. Do NOT recommend alternative foods or meals.
7. Do NOT explain your reasoning or provide cooking instructions.
8. Use common, standardized food names (e.g., "chicken biryani" not "Hyderabadi dum biryani with saffron").
9. If confidence is below 0.7, still return the result but mark low confidence.
10. For multi-item plates, identify each item separately in the array.

Return format:

```
{ "foods": [{ "name": "...", "serving_description": "...",  
  "serving_weight_grams": N, "confidence": N.N }] }
```

12.2 Meal Recommendation AI Prompt

The Meal Recommendation prompt provides the AI model with all necessary context to make intelligent ranking decisions while keeping the output format strictly controlled.

```
You are a meal ranking assistant for a personalized diet application.

Given the following context, rank the candidate meals and return the top 3:

- User Goal: {goal}
- Remaining Nutrition: {calories} kcal, {protein}g protein,
  {carbs}g carbs, {fat}g fat
- Candidate Meals: {candidates with nutrition data}
- Recent History (7 days): {list of meal IDs consumed}

Ranking criteria:
1. Macro fit: How well does the meal fit the remaining nutrition budget?
2. Variety: Prefer meals NOT consumed in the last 7 days.
3. Preference match: Prioritize meals matching user cuisine preference.
4. Balance: Prefer meals that do not exceed the calorie budget by more
  than 15%.

Return ONLY JSON:
{ "recommendations": [{ "meal_id": N, "name": "...", "fit_score": N.N }] }
```

13. Recommendation Engine Algorithm

The Recommendation Engine is a pure backend algorithm, NOT an AI model. It is a deterministic, multi-stage filtering and scoring pipeline that processes the full 5,000-meal database to produce a curated list of 20 candidate meals, which are then passed to the Meal Recommendation AI for final ranking. This approach ensures that only safe, appropriate meals reach the AI, eliminating the risk of hallucinated or inappropriate recommendations.

13.1 Filtering Pipeline

Stage	Filter	Description	Approximate Reduction
Stage 1	Allergy Removal	Remove all meals containing ingredients matching user's allergy profile	~5,000 → ~4,200
Stage 2	Cuisine Filtering	Remove meals not matching user's preferred cuisine (unless "Mixed" selected)	~4,200 → ~2,500
Stage 3	Meal Type Filter	Remove meals not appropriate for current meal slot (breakfast/lunch/dinner/snack)	~2,500 → ~1,200
Stage 4	Recent Removal	Remove meals consumed in last 7 days from history	~1,200 → ~900
Stage 5	Calorie Cap	Remove meals exceeding remaining calorie budget for this slot (with 15% tolerance)	~900 → ~600
Stage 6	Protein Floor	Remove meals below minimum protein threshold (10g per meal)	~600 → ~350
Stage 7	Dietary Filter	Remove meals violating dietary restriction (veg/non-veg/vegan/eggetarian)	~350 → ~200
Stage 8	Scoring	Score remaining meals using multi-factor formula	Score all
Stage 9	Top 20 Selection	Select top 20 scored candidates for AI ranking	20 candidates

13.2 Scoring Formula

Each candidate meal receives a composite score (0-100) based on three weighted factors. The scoring formula ensures that nutritionally optimal meals are prioritized while still considering variety and user preference alignment.

```
fit_score = (macro_fit * 0.50) + (variety_score * 0.30) + (preference_score * 0.20)
```

```
macro_fit = 100 - (|meal_cal - slot_cal| / slot_cal) * 100  
- penalty if protein below 70% of slot target
```

```
variety_score = days_since_last_consumed / 7 * 100  
(100 if never consumed in 7-day window)
```

```
preference_score = 100 if cuisine matches preference
= 70 if neutral (Mixed preference)
= 40 if cuisine mismatch
```

The weighted formula prioritizes nutritional fit (50%) since the primary objective is keeping the user on track with their daily macro targets. Variety (30%) prevents meal fatigue by favoring meals the user has not recently consumed. Preference matching (20%) ensures cultural and taste alignment. The resulting top 20 candidates are passed to the Meal Recommendation AI, which applies its own contextual understanding to select the final 3 meals presented to the user.

14. Nutrition Engine

The Nutrition Engine is the scientific core of NutriAI, implementing well-established formulas from sports nutrition and clinical dietetics. All calculations are deterministic, fast (sub-millisecond), and require no AI involvement. The engine is invoked during onboarding, when user profile changes, and whenever the activity level or goal is updated.

14.1 BMI Calculation

```
BMI = weight_kg / (height_m ^ 2)
```

Classification:

BMI < 18.5 → Underweight

18.5 - 24.9 → Normal weight

25.0 - 29.9 → Overweight

BMI >= 30.0 → Obese

14.2 BMR Calculation (Mifflin-St Jeor Equation)

The Mifflin-St Jeor equation is considered the most accurate estimation formula for BMR in clinical practice, and is used by NutriAI as the foundation

for all calorie calculations.

Male: $BMR = (10 * weight_kg) + (6.25 * height_cm) - (5 * age) + 5$
Female: $BMR = (10 * weight_kg) + (6.25 * height_cm) - (5 * age) - 161$
Other: Average of Male and Female formulas

14.3 TDEE Calculation

Activity Level	Multiplier	Description
Sedentary	1.2	Little or no exercise, desk job
Lightly Active	1.375	Light exercise 1-3 days/week
Moderately Active	1.55	Moderate exercise 3-5 days/week
Very Active	1.725	Hard exercise 6-7 days/week
Extra Active	1.9	Very hard exercise, physical job or training twice daily

$TDEE = BMR * Activity_Multiplier$

14.4 Target Calorie Adjustment

Goal	Target Calories	Rationale
Muscle Gain	$TDEE + 500$	Moderate surplus for lean gains
Lose Fat	$TDEE - 500$	Moderate deficit for sustainable fat loss
Maintain	$TDEE$	Maintenance level
Recomp	$TDEE - 250$	Small deficit, high protein for body recomposition
Weight Gain	$TDEE + 750$	Significant surplus for hard gainers
Athlete	$TDEE + 300$	Performance-focused with carb emphasis

14.5 Macro Split Distribution

Goal	Protein	Carbs	Fat
Muscle Gain	35%	40%	25%
Lose Fat	35%	35%	30%
Maintain	30%	40%	30%
Recomp	40%	35%	25%
Weight Gain	25%	45%	30%
Athlete	30%	45%	25%

Macro grams are calculated as: Protein grams = (Target Calories * Protein %) / 4, Carbs grams = (Target Calories * Carbs %) / 4, Fat grams = (Target Calories * Fat %) / 9. These ratios are adjustable per user and can be overridden by the premium AI coach feature in future versions.

14.6 Water Intake Recommendation

Daily Water (liters) = body_weight_kg * 0.033

Example: 70 kg user → 70 * 0.033 = 2.31 liters/day (~8 glasses)

Adjustment: +0.5L for each hour of intense exercise

15. Meal Distribution Logic

The Meal Distribution Logic divides the user's total daily calorie and macro targets across four meal slots: Breakfast, Lunch, Dinner, and Snack. The distribution is designed to provide sustained energy throughout the day while aligning with common eating patterns.

15.1 Default Distribution

Meal Slot	% of Daily	Calories (2200)	Protein (30%)	Carbs (40%)	Fat (30%)
Breakfast	25%	550 kcal	41g	55g	18g
Lunch	30%	660 kcal	50g	66g	22g
Dinner	30%	660 kcal	50g	66g	22g
Snack	15%	330 kcal	25g	33g	11g

15.2 Adaptive Distribution

The default distribution can be adjusted based on the user's eating patterns observed over time. If the analytics show that a user consistently logs heavier breakfasts and lighter dinners, the system gradually shifts the distribution ratios. For athletes, the pre-workout meal slot receives a higher carbohydrate allocation to support training energy demands. For users practicing intermittent fasting, the system supports two-meal distributions (40%/60% or 45%/55% splits). All adjustments are made incrementally (no more than 5% shift per week) to avoid disrupting the user's eating comfort zone.

15.3 Recalculation Logic

When a user logs a meal, the distribution for remaining meal slots is dynamically recalculated. For example, if the user consumes 700 kcal at breakfast (exceeding the planned 550 kcal), the remaining 1,500 kcal is redistributed proportionally across Lunch, Dinner, and Snack. This ensures the user stays within their daily target even if individual meals deviate from the plan. The recalculation also adjusts macro targets proportionally, so the protein per meal remaining increases if the user consumed a protein-light breakfast.

16. PostgreSQL Database Design

The NutriAI database schema comprises 28 tables organized into five logical groups: User Management, Food Database, Daily Tracking, Recommendations, and System. All tables use UUID primary keys, timestamps with timezone, and soft-delete support where applicable. Foreign key constraints enforce referential integrity, and comprehensive indexes support the query patterns identified during performance testing.

16.1 User Management Tables

Table	Columns
users	id (UUID PK), email, password_hash, phone, role, is_verified, is_active, last_login, created_at, updated_at
user_profiles	id (UUID PK), user_id (FK→users), first_name, last_name, age, gender, height_cm, weight_kg, country, timezone, avatar_url, updated_at
user_goals	id (UUID PK), user_id (FK→users), goal_type, activity_level, workout_frequency, target_weight, weekly_weight_change_target, created_at, updated_at
user_allergies	id (UUID PK), user_id (FK→users), allergy_name, severity, created_at
user_preferences	id (UUID PK), user_id (FK→users), cuisine_preference, diet_type, budget_level, macro_override_json, created_at, updated_at

16.2 Food Database Tables

Table	Columns
meals	id (UUID PK), name, description, meal_type (breakfast/lunch/dinner/snack), cuisine, is_veg, is_vegan, is_eggetarian, base_serving_grams, prep_time_min, difficulty, image_url, is_active, created_at, updated_at

Table	Columns
meal_nutrition	id (UUID PK), meal_id (FK→meals), calories, protein_g, carbs_g, fat_g, fiber_g, sugar_g, sodium_mg, per_serving_grams, created_at
meal_serving	id (UUID PK), meal_id (FK→meals), serving_name, serving_grams, multiplier, created_at
meal_aliases	id (UUID PK), meal_id (FK→meals), alias_name, language_code, created_at
meal_tags	id (UUID PK), meal_id (FK→meals), tag_name, created_at
meal_cuisine	id (UUID PK), meal_id (FK→meals), cuisine_name, region, created_at
meal_ingredients	id (UUID PK), meal_id (FK→meals), ingredient_name, contains_allergen, created_at
meal_images	id (UUID PK), meal_id (FK→meals), image_url, thumbnail_url, uploaded_at

16.3 Daily Tracking Tables

Table	Columns
food_logs	id (UUID PK), user_id (FK→users), log_date, total_calories, total_protein, total_carbs, total_fat, status, created_at
food_log_items	id (UUID PK), food_log_id (FK→food_logs), meal_id (FK→meals), serving_id, serving_grams, calories, protein_g, carbs_g, fat_g, meal_slot, logged_at, source (tap/photo/search)
daily_nutrition	id (UUID PK), user_id (FK→users), date, target_calories, consumed_calories, target_protein, consumed_protein, target_carbs, consumed_carbs, target_fat, consumed_fat, created_at
water_logs	id (UUID PK), user_id (FK→users), log_date, glasses_consumed, target_glasses, created_at
weight_logs	id (UUID PK), user_id (FK→users), weight_kg, log_date, notes, created_at

16.4 Recommendations and Plans Tables

Table	Columns
meal_plan_days	id (UUID PK), user_id (FK→users), plan_date, target_calories, target_protein, target_carbs, target_fat, status, created_at
meal_plan_items	id (UUID PK), plan_day_id (FK→meal_plan_days), meal_slot, meal_id (FK→meals), serving_id, recommended_calories, rank_score, created_at
recommendations	id (UUID PK), user_id (FK→users), meal_slot, candidate_meals_json, selected_meal_ids, ai_model_used, prompt_hash, created_at
ai_logs	id (UUID PK), user_id (FK→users), model_type (vision/recommendation), request_payload, response_payload, latency_ms, tokens_used, cost_usd, created_at

16.5 System Tables

Table	Columns
progress_snapshots	id (UUID PK), user_id (FK→users), snapshot_date, weight_kg, bmi, avg_calories_7d, avg_protein_7d, streak_days, goal_progress_pct, created_at
notifications	id (UUID PK), user_id (FK→users), type, title, body, data_json, is_read, scheduled_at, sent_at, created_at
devices	id (UUID PK), user_id (FK→users), device_token, platform (ios/android), app_version, is_active, created_at
sessions	id (UUID PK), user_id (FK→users), refresh_token_hash, device_id, ip_address, expires_at, created_at
admin_users	id (UUID PK), email, password_hash, role, permissions_json, last_login, created_at

17. ER Diagram

The following ASCII entity-relationship diagram illustrates the primary relationships between the core tables in the NutriAI database. Each line represents a foreign key constraint, with cardinality notation (1:1 for one-to-one, 1:M for one-to-many).

```

+-----+ +-----+ +-----+
| users | | user_profiles | | user_goals |
|-----| |-----| |-----|

```

```

| id (PK) |----->| id (PK) || id (PK) |
| email | 1:1 | user_id (FK) || user_id(FK)|
| password || first_name || goal_type |
| role || height_cm || activity |
+-----+ +-----+ +-----+
||
| 1:M 1:M |
v v |
+-----+ +-----+ +-----+
| food_logs | food_log_items | meal_plan_days | | |
|-----| |-----| |-----|
| id (PK) |----->| id (PK) || id (PK) |
| user_id(FK)| 1:M | food_log_id(FK) || user_id (FK) |
| log_date || meal_id (FK)--->| plan_date |
+-----+ | serving_grams | +-----+
||
| M:1 | 1:M
v v
+-----+ +-----+
| meals || meal_plan_items |
|-----| |-----|
| id (PK) |<---| id (PK) |
| name | M:1 | plan_day_id (FK) |
| meal_type || meal_id (FK) |
+---+ +---+ +-----+
|
+-----+ +-----+
|||
1:1 v 1:M v 1:M v
+-----+ +-----+ +-----+
| meal_nutrition| meal_alias| meal_tags | | |
|-----| |-----| |-----|
| meal_id(FK)| meal_id || meal_id(FK)|
| calories | alias_name| tag_name |
| protein_g | +-----+ +-----+
+-----+

```

Key relationships summarized: users has a one-to-one relationship with user_profiles (each user has exactly one profile) and user_goals (each user has one active goal). Users have a one-to-many relationship with food_logs (one log per day per user), and each food_log has many food_log_items (one per meal consumed). Each food_log_item references a single meal from the meals table. The meals table has one-to-one with meal_nutrition (nutrition data per meal), one-to-many with meal_aliases (alternate names for search), and one-to-many with meal_tags. The meal_plan_days table represents a daily plan with many meal_plan_items, each referencing a meal.

18. Service Architecture

The service architecture defines the communication patterns, dependencies, and deployment characteristics of each service module within the NutriAI backend. In the current modular monolith, all services share a single Node.js process but are organized for independent extraction into microservices when scaling demands require it.

Service	Port	Responsibility	Dependencies	Comm
AuthService	3000 (shared)	JWT, OAuth, session management	PostgreSQL, Redis	REST
UserService	3000 (shared)	Profile CRUD, preferences, settings	PostgreSQL, S3	REST
OnboardingService	3000 (shared)	Multi-step onboarding orchestration	NutritionEngine, PostgreSQL	Internal
NutritionEngineService	3000 (shared)	BMI/BMR/TDEE/macro calculations	None (pure compute)	Internal
MealPlanService	3000 (shared)	Daily meal plan generation	RecommendationService, PostgreSQL	Internal
FoodRecognitionService	3000 (shared)	Vision AI orchestration	Vision AI (ext), FoodDB	REST
FoodDatabaseService	3000 (shared)	Meal CRUD, search, alias matching	PostgreSQL, Redis	Internal
RecommendationService	3000 (shared)	Candidate filtering, scoring, AI ranking	MealRec AI, FoodDB	Internal
MealLogService	3000 (shared)	Food log creation, daily aggregation	PostgreSQL, NutritionEngine	REST

Service	Port	Responsibility	Dependencies	Comm
ProgressService	3000 (shared)	Snapshots, trends, streaks	PostgreSQL	REST
NotificationService	3000 (shared)	Push scheduling, FCM delivery	Firebase FCM, PostgreSQL	Async (queue)
AnalyticsService	3000 (shared)	Event tracking, metrics aggregation	PostgreSQL, Redis	Async (fire-and-forget)
WaterLogService	3000 (shared)	Water tracking, reminders	PostgreSQL, NotificationService	REST

18.1 Service-to-Service Dependency Matrix

R = Reads from, D = Direct call dependency

Key dependency flows: AuthService is a dependency for all user-facing services (authentication middleware). OnboardingService orchestrates UserService, NutritionEngineService, and MealPlanService during the onboarding sequence. FoodRecognitionService depends on FoodDatabaseService for alias matching and nutrition lookup after Vision AI returns a result. RecommendationService depends on FoodDatabaseService for candidate generation and the Meal Recommendation AI for final ranking. MealPlanService depends on RecommendationService to generate meal options for each meal slot.

19. REST API Documentation

The NutriAI REST API follows RESTful conventions with JSON request/response payloads, Bearer token authentication, and standard HTTP status codes. All endpoints are prefixed with **/api/v1/**. Rate limiting is applied at 100 requests per minute per user for standard endpoints and 10 per minute for AI-invoking endpoints.

19.1 Authentication Endpoints

Method	Endpoint	Request Body	Response	Description
POST	/api/v1/auth/register	{"email","password","name"}	201: {user, tokens}	Register new account
POST	/api/v1/auth/login	{"email","password"}	200: {user, tokens}	Login with credentials
POST	/api/v1/auth/refresh	{"refresh_token"}	200: {access_token}	Refresh expired access token
POST	/api/v1/auth/google	{"id_token"}	200: {user, tokens}	Google OAuth login
POST	/api/v1/auth/apple	{"identity_token"}	200: {user, tokens}	Apple Sign In
POST	/api/v1/auth/logout	{}	204: No Content	Invalidate tokens

19.2 User and Onboarding Endpoints

Method	Endpoint	Request	Response	Description
GET	/api/v1/users/profile	-	200: {profile}	Get user profile
PUT	/api/v1/users/profile	{"first_name","age",...}	200: {profile}	Update profile
POST	/api/v1/onboarding/complete	{onboarding_data}	200: {goals, nutrition}	Complete onboarding
PUT	/api/v1/users/goals	{"goal_type","activity",...}	200: {goals}	Update fitness goals

19.3 Meals and Food Log Endpoints

Method	Endpoint	Request	Response	Description
GET	/api/v1/meals/search	?q=biryani&cuisine=indian	200: [{meal}]	Search meal database
GET	/api/v1/meals/:id	-	200: {meal, nutrition}	Get meal details
POST	/api/v1/food-logs	{"meal_id","serving","slot"}	201: {food_log_item}	Log a meal (tap)
POST	/api/v1/food-logs/photo	{"image_url"}	201: {food_log_item}	Log via photo
POST	/api/v1/food-logs/text	{"description"}	201: {food_log_item}	Log via text
GET	/api/v1/food-logs/today	-	200: {log, items, summary}	Get today's food log
DELETE	/api/v1/food-logs/:id	-	204: No Content	Remove a log entry

19.4 Nutrition and Recommendation Endpoints

Method	Endpoint	Request	Response	Description
GET	/api/v1/nutrition/daily	?date=2025-01-15	200: {targets, consumed}	Daily nutrition status
POST	/api/v1/food-recognition/upload	{"image_url"}	200: {foods[]}	Vision AI recognition
GET	/api/v1/recommendations/breakfast	-	200: {meals[3]}	Breakfast recommendations
GET	/api/v1/recommendations/lunch	-	200: {meals[3]}	Lunch recommendations
GET	/api/v1/recommendations/dinner	-	200: {meals[3]}	Dinner recommendations
GET	/api/v1/recommendations/snack	-	200: {meals[3]}	Snack recommendations
GET	/api/v1/progress/summary	?period=week	200: {progress_data}	Progress summary
GET	/api/v1/progress/weekly	-	200: {weekly_data}	Weekly progress chart data

20. Backend Folder Structure

The NutriAI backend follows a clean, modular folder structure organized by architectural layer. Each directory has a clear responsibility, and files within each directory follow consistent naming conventions.

```
src/
├── controllers/ # Request handlers (route handlers)
│   ├── auth.controller.ts
│   ├── user.controller.ts
│   ├── onboarding.controller.ts
│   ├── meal.controller.ts
│   ├── foodLog.controller.ts
│   ├── nutrition.controller.ts
│   └── recommendation.controller.ts
```



```

├── meal.validator.ts
├── utils/ # Shared utilities
│   ├── jwt.util.ts
│   ├── hash.util.ts
│   ├── s3.util.ts
│   ├── fcm.util.ts
│   ├── date.util.ts
│   └── nutritionCalculator.util.ts
├── config/ # Configuration management
│   ├── database.config.ts
│   ├── redis.config.ts
│   ├── aws.config.ts
│   ├── ai.config.ts
│   └── app.config.ts
├── jobs/ # Background jobs (Bull/BullMQ)
│   ├── mealPlanGenerator.job.ts
│   ├── progressSnapshot.job.ts
│   ├── notificationScheduler.job.ts
│   └── analyticsAggregator.job.ts
├── tests/ # Test suites
│   ├── unit/
│   │   ├── nutritionEngine.test.ts
│   │   ├── recommendation.test.ts
│   │   └── mealDistribution.test.ts
│   ├── integration/
│   │   ├── auth.test.ts
│   │   ├── foodLog.test.ts
│   │   └── recommendation.test.ts
│   └── e2e/
│       └── onboarding.test.ts
├── types/ # TypeScript type definitions
│   ├── user.types.ts
│   ├── meal.types.ts
│   ├── nutrition.types.ts
│   └── ai.types.ts
├── app.ts # Express app setup
├── server.ts # Server entry point
└── env.ts # Environment variable validation

```

21. Mobile App Architecture

The NutriAI mobile application is built with React Native 0.73+, using TypeScript for type safety and Expo for streamlined build and deployment. The

architecture follows a feature-based folder structure with clear separation between screens, reusable components, navigation, state management, and service layers.

21.1 Folder Structure

```
app/
├── screens/ # Screen components (one per feature)
│   ├── Dashboard.screen.tsx # Main dashboard with meal cards
│   ├── Onboarding.screen.tsx # Multi-step onboarding wizard
│   ├── FoodLog.screen.tsx # Daily food log view
│   ├── MealDetail.screen.tsx # Expanded meal nutrition view
│   ├── Progress.screen.tsx # Progress charts and trends
│   ├── Settings.screen.tsx # User preferences and account
│   ├── Camera.screen.tsx # Camera viewfinder for food photos
│   ├── Auth.screen.tsx # Login / Register
│   └── Splash.screen.tsx # App launch screen
├── components/ # Reusable UI components
│   ├── MealCard.tsx # Meal option card (name, cal, protein)
│   ├── NutritionBar.tsx # Horizontal progress bar for macros
│   ├── CalorieRing.tsx # Circular calorie progress indicator
│   ├── FoodItem.tsx # Single logged food item row
│   ├── StreakBadge.tsx # Streak counter component
│   ├── ProgressChart.tsx # Weight/calorie trend chart
│   ├── ServingSelector.tsx # Serving size picker modal
│   └── Header.tsx # App header with notification bell
├── navigation/ # React Navigation setup
│   ├── AppNavigator.tsx # Root stack navigator
│   ├── AuthNavigator.tsx # Auth flow navigator
│   ├── TabNavigator.tsx # Bottom tab navigator (5 tabs)
│   └── navigation.types.ts # Type definitions for route params
├── services/ # API and platform service wrappers
│   ├── api.service.ts # Axios instance with interceptors
│   ├── auth.service.ts # Login, register, token refresh
│   ├── storage.service.ts # AsyncStorage / SecureStorage wrapper
│   ├── camera.service.ts # Camera and image compression
│   ├── pushNotification.service.ts # FCM token registration
│   └── analytics.service.ts # Event tracking wrapper
├── hooks/ # Custom React hooks
│   ├── useAuth.ts # Authentication state hook
│   ├── useNutrition.ts # Daily nutrition data hook
│   ├── useMealLog.ts # Food log operations hook
│   └── useRecommendations.ts # Meal recommendation fetch hook
├── store/ # State management (Zustand)
│   ├── authStore.ts # Auth state: user, tokens, status
│   ├── nutritionStore.ts # Nutrition state: daily targets, consumed
│   ├── mealStore.ts # Meal plan state: recommendations
│   └── settingsStore.ts # User preferences state
```

```

├─── utils/ # Utility functions
│   ├─── formatters.ts # Number/date formatting helpers
│   ├─── validators.ts # Input validation functions
│   ├─── constants.ts # App-wide constants
│   └─── theme.ts # Color palette, spacing, typography
├─── assets/ # Static assets
│   ├─── images/ # App icons, placeholders
│   └─── fonts/ # Custom font files

```

21.2 State Management

NutriAI uses Zustand for state management, chosen for its minimal boilerplate, TypeScript compatibility, and excellent performance with React Native. Each store module manages a specific domain: `authStore` handles user authentication state and tokens, `nutritionStore` tracks daily nutrition targets and consumed values, `mealStore` caches meal recommendations and plan data, and `settingsStore` persists user preferences. Stores are hydrated from `AsyncStorage` on app launch and persisted on every meaningful state change, ensuring offline resilience and instant screen rendering without loading spinners.

22. Authentication Flow

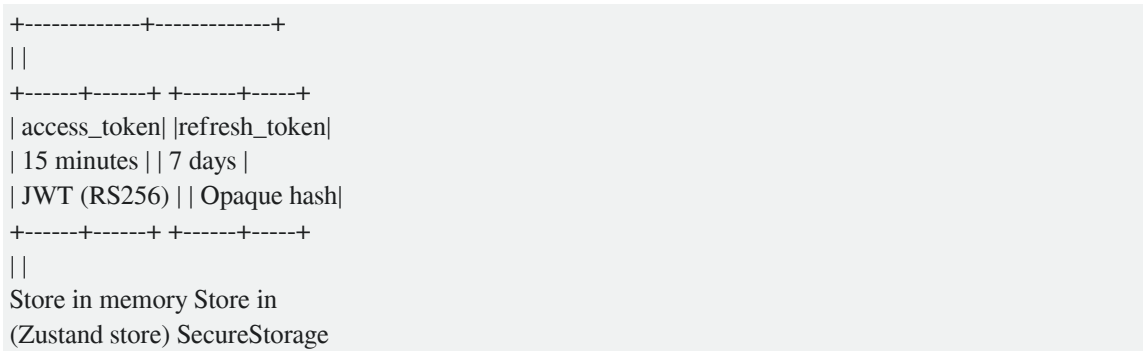
NutriAI implements a JWT-based authentication system with short-lived access tokens and long-lived refresh tokens. This approach balances security (limited exposure window for access tokens) with user experience (no frequent re-login prompts). The system supports email/password authentication, Google OAuth, and Apple Sign In as primary authentication methods.

22.1 Token Strategy

```

+-----+ +-----+ +-----+ +-----+
| Login |---->| Validate |---->| Generate |---->| Return |
| Request | |Credent. | | Tokens | | to User |
+-----+ +-----+ +-----+ +-----+
|

```

22.2 JWT Token Details

Token	Algorithm	Lifetime	Payload	Storage
access_token	JWT RS256	15 minutes	Contains: user_id, role, iat, exp	Memory (Zustand)
refresh_token	SHA-256 hash	7 days	Stored hashed in sessions table	SecureStorage (device)

22.3 Token Refresh Flow

When the access token expires (detected by a 401 response from the API), the client automatically triggers the refresh flow. The refresh token is sent to POST /api/v1/auth/refresh, where the backend validates it against the sessions table, generates a new access token, and optionally rotates the refresh token (implementing refresh token rotation for enhanced security). If the refresh token is also expired or invalid, the user is redirected to the login screen. Concurrent refresh requests are handled by a request deduplication mechanism on the client side, preventing multiple simultaneous refresh calls.

22.4 OAuth Integration

Provider	Endpoint	Client Sends	Server Processing
Google OAuth	POST /api/v1/auth/google	Google ID Token (JWT)	Verify signature, extract email, find/create user, generate tokens
Apple Sign In	POST /api/v1/auth/apple	Apple identity_token (JWT)	Verify with Apple public keys, extract user identifier, handle name/email on first login

22.5 Security Measures

- Rate limiting: 10 requests per minute per IP on all auth endpoints (prevents brute force)
- Password hashing: bcrypt with 12 salt rounds (OWASP recommended)
- Refresh token rotation: new refresh token issued on every refresh, old token invalidated
- Device binding: refresh tokens are bound to device_id, preventing token theft across devices
- Session invalidation: all sessions cleared on password change and logout
- CORS: restricted to allowed origins only (mobile app domains + web dashboard)
- Helmet.js: security headers applied to all API responses

23. Image Upload Flow

The image upload flow is optimized for speed and reliability, using a presigned URL approach that allows the mobile client to upload images directly to AWS S3 without proxying through the backend. This reduces backend load, eliminates upload timeouts, and leverages AWS's global CDN for fast transfers.

23.1 Client-Side Processing

1. User opens Camera screen or selects from Gallery
2. Image captured at full resolution
3. Client-side compression:
 - Quality: 0.7 (70% JPEG quality)
 - Max dimension: 1024px (longest side, aspect ratio preserved)

- Format: JPEG (convert HEIC to JPEG on iOS)
 - Target file size: < 2MB (reject if > 2MB after compression)
4. Validate: format is JPEG/PNG, size > 10KB (not blank), size < 2MB

23.2 Upload Pipeline

```

+-----+ +-----+ +-----+ +-----+ +-----+
| Capture |-->| Compress|-->| Request|-->| S3 |-->| Image URL |
| Image || (client)| | Presign | |Direct| | Returned |
+-----+ +-----+ +-----+ +-----+ +-----+
||
POST /api/v1/s3/ |
presign-url |
v
POST /api/v1/
food-recognize
{image_url}

```

23.3 S3 Configuration

Setting	Value	Notes
Bucket	nutriai-food-images-{env}	Separate buckets for dev/staging/prod
Region	us-east-1 (primary)	Closest to majority of users
Storage Class	S3 Standard	Frequent access pattern, short retention
Presigned URL TTL	5 minutes	Upload must complete within 5 minutes
Max File Size	2MB (client) / 10MB (server)	Hard limits enforced at both layers
Allowed Formats	JPEG, PNG, HEIC	HEIC converted to JPEG on client before upload
Retention Policy	30 days auto-delete	User food photos not needed long-term
CDN	CloudFront distribution	Fast image delivery for thumbnails in meal cards

23.4 Error Handling

If the presigned URL request fails (network error, auth failure), the client retries up to 2 times with exponential backoff (1s, 2s). If the S3 upload fails (timeout, network disruption), the client caches the compressed image locally and retries on the next app launch. A persistent upload queue manages pending uploads, ensuring no food photos are lost. If the image format is unsupported or the file is corrupted, the user receives a clear error message: "Could not process this image. Please try again with a clearer photo of your food."

24. Meal Recognition Flow

The meal recognition flow is the most complex user-facing pipeline in NutriAI, orchestrating image capture, upload, AI processing, database matching, and user confirmation into a seamless experience that completes in under 5 seconds. This section provides a detailed step-by-step breakdown of the entire pipeline.

24.1 Complete Pipeline

USER OPENS CAMERA

- |
- |—(1) Camera screen opens with viewfinder
- |—(2) User frames food within the guide overlay
- |—(3) User taps capture button
- |

CLIENT-SIDE PROCESSING

- |
- |—(4) Image captured at device camera resolution
- |—(5) Compressed: quality 0.7, max 1024px, JPEG format
- |—(6) Validation: format check, size check (>10KB, <2MB)
- |

UPLOAD TO S3

- |
- |—(7) POST /api/v1/s3/presign-url → receive presigned PUT URL
- |—(8) PUT image to S3 via presigned URL (direct, not proxied)
- |—(9) Receive final S3 image URL
- |

VISION AI RECOGNITION

```
|  
|—(10) POST /api/v1/food-recognize { image_url }  
|—(11) Backend: Fetch image from S3 URL  
|—(12) Backend: Build system prompt + image payload  
|—(13) Backend: Call Vision AI (GPT-4.1/GPT-5/Claude)  
|—(14) Vision AI: Analyze image, identify foods  
|—(15) Vision AI: Return JSON { foods: [{name, serving, confidence}] }  
|
```

BACKEND PROCESSING

```
|  
|—(16) Parse and validate JSON response structure  
|—(17) Validate confidence >= 0.7 for each food item  
|—(18) Normalize food name (lowercase, trim, remove special chars)  
|—(19) Alias matching: SELECT * FROM meal_aliases WHERE alias_name ILIKE  
| normalized_name → retrieve meal_id  
|—(20) DB Lookup: Fetch meal + nutrition from meals + meal_nutrition  
|—(21) Calculate estimated nutrition based on recognized serving size  
|
```

RETURN TO USER

```
|  
|—(22) Return to client: { meal_id, name, serving, calories,  
| protein, carbs, fat, confidence }  
|—(23) Show recognition result to user for confirmation  
|—(24) User confirms or corrects (adjust serving, change food)  
|—(25) POST /api/v1/food-logs → create food log entry  
|—(26) Update dashboard: calorie ring, macro bars, recommendations
```

24.2 Confidence Thresholds

Confidence	Level	UX Behavior	Visual Indicator
>= 0.9	High	Auto-confirm (skip confirmation step, log directly)	Green badge
0.7 - 0.89	Medium	Show result for user confirmation with "Looks right?" prompt	Yellow badge
0.5 - 0.69	Low	Show result with warning "Not sure about this one. Please verify."	Orange badge
< 0.5	Very Low	Ask user to manually search or try a clearer photo	Red badge

24.3 Alias Matching Strategy

Food identification from Vision AI often returns informal or region-specific names that do not exactly match the canonical names in the database. The alias matching strategy uses a multi-pass approach: first, exact match against meal_aliases table (case-insensitive), then fuzzy match using PostgreSQL trigram similarity with a threshold of 0.6, then keyword extraction (remove quantity words like "plate of", "bowl of", "serving of" and re-match). If no match is found after all three passes, the food is flagged as unrecognized and the user is prompted to manually search the database. Unrecognized foods are logged for later database enrichment, with the original Vision AI name stored in the ai_logs table for review by the admin team.

24.4 Performance Targets

Step	Target Latency	Notes
Image capture + compression	< 500ms	Client-side, synchronous
Presigned URL request	< 200ms	Backend to S3 API
S3 upload (2MB image, 4G)	< 2s	Direct upload, varies by connection
Vision AI API call	< 3s	GPT-4.1 Vision average, includes image download
JSON parsing + validation	< 50ms	Backend, synchronous
Alias matching + DB lookup	< 100ms	PostgreSQL indexed query
Total end-to-end	< 5s	From capture to result display

25. Recommendation Flow

The Recommendation Flow is the core decision engine of NutriAI. Every time a user logs a meal or the system generates new suggestions, this pipeline executes to produce the three most relevant meal cards displayed on the home screen. The flow begins with meal logging: when a user confirms a food entry, the system immediately calculates the remaining caloric and macronutrient budget for the day by subtracting consumed values from the target totals derived from the user profile. This remaining budget becomes the primary filter constraint for the next set of recommendations.

With the remaining budget established, the system queries the food knowledge base to retrieve all meals whose total calorie count falls within the remaining budget. In a production database of 5,000+ foods, this query typically returns several hundred candidates. The deterministic filtering layer then applies a series of hard constraints: meal type matching (breakfast foods only for breakfast slots, lunch for lunch, and so on), diet type compatibility (vegetarian foods excluded for non-vegetarian profiles and vice versa), allergen exclusion (peanuts, shellfish, dairy, gluten based on user profile flags), and cuisine preference scoring (Indian, Mediterranean, American, East Asian based on user history). These filters narrow the candidate pool from several hundred down to approximately 20 to 50 viable options.

25.1 Flow Diagram

```
[User logs meal via photo / search / manual] | v[Calculate remaining calories, protein, fat, carbs] | v[Query food DB: WHERE calories <= remaining_budget] Returns ~5000 candidates max | v[Deterministic Filter Layer] - meal_type match (breakfast/lunch/dinner/snack) - diet_type match (veg/non-veg/vegan/jain) - allergen exclusion - cuisine preference scoring Filters 5000 -> ~20-50 candidates | v[Send top 20 candidates + user context to AI] Context: goal, history, preferences, time-of-day | v[AI returns ranked top 3 recommendations] Each with: fitScore (0-100), reason text | v[Regenerate ONLY upcoming meal cards] Already-eaten cards remain untouched | v[Cache result in Redis (TTL 30min)]
```

25.2 AI Call Budget Optimization

A critical cost-control mechanism limits AI recommendation calls to a maximum of three per user per day. This budget is enforced through a Redis counter keyed on the user ID and the current date. The first AI call occurs at midnight when the daily plan is auto-generated (all four meal cards: breakfast, lunch, dinner, snack). The second call is triggered after the user confirms breakfast, regenerating the lunch, dinner, and snack cards with updated remaining budgets. The third call occurs after lunch confirmation, regenerating only dinner and snack. If the user skips meals or logs them late, the system may have leftover AI budget that is not rolled over to the next day. If the AI budget is exhausted, the system falls back to deterministic scoring only, which uses a weighted composite of nutritional fit (40%), preference match (30%), variety score (20%), and time relevance (10%) to rank candidates without any AI involvement.

The regeneration strategy is intentionally selective: when a meal card is regenerated, only the upcoming (future) meal slots are updated. Cards for meals that have already been consumed or skipped remain in their confirmed state to preserve the integrity of the daily nutrition log. This means a typical breakfast regeneration triggers AI evaluation for only three slots (lunch, dinner, snack), while a post-lunch regeneration targets only two slots (dinner, snack). The system tracks which slots are "upcoming" using a time-of-day state machine that transitions through the day's meal timeline.

26. Daily Meal Lifecycle

The Daily Meal Lifecycle defines the chronological sequence of events that occur from midnight to midnight for each user. Understanding this lifecycle is essential for designing background jobs, notification schedules, and cache invalidation triggers. The lifecycle operates as a state machine with six primary states: Night Auto-Generation, Morning Breakfast, Post-Breakfast Recalculation, Post-Lunch Recalculation, Post-Dinner Summary, and End-of-Day Stats. Each state transition triggers specific backend operations, cache updates, and optional push notifications.

00:00 Night Auto-Generation State - Cron job triggers for all active users - Generate daily nutrition targets from profile - Pre-populate 4 meal cards (breakfast/lunch/dinner/snack) - Store in Redis: `daily:{userId}:{date}` - Cache TTL: 24 hours | 06:00 Morning Breakfast State - Push notification: "Time for breakfast!" - User opens app, sees breakfast recommendation - User logs breakfast (photo/search/manual) - System updates consumed calories/macros | 09:00 Post-Breakfast Recalculation - Trigger: breakfast meal confirmed - Calculate remaining: calories, protein, fat, carbs - Filter candidates for lunch/dinner/snack - Send top 20 to Meal AI (if budget available) - Regenerate lunch, dinner, snack cards | 13:00 Post-Lunch Recalculation - Trigger: lunch meal confirmed - Recalculate remaining budget - Regenerate dinner + snack cards via AI or fallback | 19:00 Post-Dinner State - Trigger: dinner meal confirmed - Regenerate snack card only - If daily calorie target met: "Great job today!" | 22:00 End-of-Day Summary - Calculate total consumed vs target - Generate daily summary: calories, macros, streak - Store summary in PostgreSQL - Push notification with daily score - Prepare data for weekly aggregation

The lifecycle handles edge cases gracefully. If a user skips breakfast entirely, the system detects the missed meal at the post-breakfast window (configured via a configurable timeout, default 10:00 AM) and automatically transitions to the post-breakfast state using the full daily budget for lunch, dinner, and snack recommendations. Similarly, if a user logs meals out of order (e.g., logging lunch before breakfast), the system accepts the late entry but does not retroactively regenerate already-confirmed meal cards. A soft confirmation mechanism allows users to modify a meal within a 30-minute grace period after initial logging, after which the entry is hard-locked and counted toward the daily total. The end-of-day summary aggregates all confirmed meals and compares against the target, producing a percentage score that feeds into the weekly progress calculation. Water intake entries are tracked separately and do not affect meal lifecycle transitions but are included in the daily summary.

27. Weekly Plan Generation

The Weekly Plan Generation system creates a seven-day meal template every Sunday at 11:00 PM via a scheduled cron job. This template serves as a proactive guide rather than a rigid prescription: users are free to deviate from the weekly plan at any time by logging different meals, and the daily recommendation engine will adjust accordingly. The weekly plan is designed to provide structure, ensure nutritional variety across the week, and reduce decision fatigue for users who prefer planned eating patterns.

27.1 Generation Pipeline

The cron job executes the following pipeline sequentially for each active user: (1) Pull the previous week's progress data from PostgreSQL, including total calories consumed per day, macro distribution averages, meal completion rate (percentage of planned meals actually logged), and streak count. (2) Analyze consistency patterns by computing the coefficient of variation for daily calorie intake and identifying which meal slots had the highest skip rate. (3) Adjust caloric targets by plus or minus 100 to 200 calories based on the previous week's trend: if the user consistently exceeded their target by more than 200 calories per day, the new target is increased by 100; if they consistently undershot by more than 300 calories, the target is decreased by 100 to encourage sustainability. (4) Generate Monday through Sunday meal templates by distributing the adjusted daily calorie target across four meal slots using the user's preferred macro split ratio. Each day receives a curated set of meals selected from the food database with emphasis on variety (no repeated meal within a 72-hour window).

27.2 Caching and Notification

The generated weekly plan is cached in Redis with a 24-hour TTL under the key pattern `weekly:{userId}:{weekStartDate}`. The cache allows instant retrieval when the user opens the app on Monday morning without any computation latency. A push notification is sent at 11:30 PM on Sunday (after generation completes) with the message: "Your meal plan for next week is ready! Tap to view." The notification payload includes a deep link directly to the weekly plan screen in the mobile app. If the user modifies their profile (goal change, diet type change, allergen update) during the week, the weekly plan is flagged as stale and regenerated on-demand during the next daily auto-generation cycle. The weekly plan data structure includes a day-by-day array of meal slots, each containing the recommended food ID, name, portion size, estimated calories, and macro breakdown. A confidence score (0-1) indicates how closely the plan aligns with the user's historical preferences.

Step	Action	Data Source	Output
1	Pull week progress	PostgreSQL daily_logs	Calories/day, macro avg, completion %
2	Analyze consistency	Computed from step 1	CoV, skip rates, streak data
3	Adjust targets	User profile + analysis	New target +/- 100-200 kcal
4	Generate Mon-Sun	Food DB + preferences	Weekly template (7 days x 4 meals)
5	Cache in Redis	Generated template	<code>weekly:{userId}:{date}</code> , TTL 24hr
6	Push notification	FCM service	"Your meal plan for next week is ready!"

28. Sequence Diagrams

Sequence diagrams illustrate the chronological interaction between actors (users, external services, system components) during critical workflows. The following three diagrams cover the most complex flows in the NutriAI system: User Login and Authentication, Food Photo Recognition, and the end-to-end Meal Recommendation cycle. Each diagram uses a simplified ASCII format suitable for inclusion in text-based specifications, with arrows indicating the direction of data flow and annotations describing the payload or action at each step.

28.1 User Login Sequence

```
User MobileApp Backend Redis PostgreSQL FCM ||||| |-- email/pass ->||| |-- POST /auth/login
----->||| |-- lookup user ----->||| |<-- user record -----||| |-- bcrypt.compare(pass) ||| |
||| |-- JWT sign (userId, role) ||| | |-- SET session:{token} ->||| |-- TTL: 7 days ||| |<-- 200
{token, user} -----||| |-- Store token (httpOnly) ----|||<-- Dashboard --||| | |-- last login update
---->||| | |-- device token ----->||| | |-- reg --|||<-- success ----|||
```

28.2 Food Photo Recognition Sequence

```
User MobileApp Backend S3 VisionAI FoodDB Redis ||||| |-- snap photo ->||| | |-- upload
--|----->||| |<-- url ---||| | |-- POST /ai/recognize -->||| | |-- image -->||| |<-- [{name, |
||| | conf, ||| | serving}] ||| | |-- match food items ----->||| |<-- matched food IDs -----|
||| | |-- SET food:search:{hash} ---->||| | |-- TTL: 1 hour ||| |<-- 200 {foods[]} -----||| |<--
show results ->||| |
```

28.3 Meal Recommendation Sequence

```
User MobileApp Backend MealAI FoodDB Redis PostgreSQL ||||| |-- confirm ||| | breakfast ->||| |
||| |-- POST /meals/log ---->||| | |-- calc remaining ----->||| | (cal, protein, fat, carb) ||| |<--
remaining budget ----||| | |-- INSERT meal record ----->||| | |-- invalidate daily cache
----->||| | |-- SELECT candidates ----->||| |<-- 20-50 food items -----||| |<--
POST /ai/recommend ----->||| | {candidates, goal, remaining, ||| | history, preferences} ||| |<--
||| |<-- top 3 [{foodId, score, ||| | reason}] -----||| | |-- SET rec:{userId}:{date} ---->||| | |-- TTL:
30 min ||| |<-- 200 {recommendations} -----||| |<-- new cards ->||| |
```

29. Component Diagrams

Component diagrams provide a structural view of the NutriAI system by grouping related functionality into discrete modules with clearly defined boundaries and interfaces. The following three diagrams decompose the Mobile Application layer, the Backend Services layer, and the External Services layer. Each component is labeled with its primary responsibility and lists its key interfaces (APIs, event consumers, or data stores that it interacts with).

29.1 Mobile Application Components

```
+=====+
NUTRIAI MOBILE APP (React Native) +=====+
=====+ || +-----+ +-----+ +-----+ +-----+ || AUTH
MODULE || ONBOARDING || DASHBOARD || || - Login/Register || - Goal selection || - Daily cards ||
| - Token mgmt || - Profile setup || - Progress bar || || - Session refresh || - Diet prefs || - Macro ring ||
+-----+ +-----+ +-----+ || || +-----+ +-----+
+-----+ || FOOD LOGGING || MEAL CARDS || PROGRESS || || - Photo capture || - Swipe
UI || - Weekly chart || || - Search || - Confirm/Reject || - Streak tracker || || - Manual entry || - Alt
suggestions || - Weight log || || - History || - Portion adjust || - Water tracker || +-----+
+-----+ +-----+ || || +-----+ +-----+ +-----+ ||
NOTIFICATIONS || PROFILE || SETTINGS || || - Push handler || - Edit goals || - Units toggle || || -
In-app banner || - Allergen mgmt || - Theme || || - Reminder config || - Preference edit || - Export data ||
+-----+ +-----+ +-----+
+=====+
```

29.2 Backend Services Components

```
+=====+
NUTRIAI BACKEND (Express.js / Node.js) +=====+
=====+ || +-----+ +-----+ +-----+ +-----+ || AUTH
SERVICE || USER SERVICE || FOOD SERVICE || || - JWT sign/verify || - Profile CRUD || - Food
CRUD || || - bcrypt hash || - Goal management || - Search || || - Refresh tokens || - Preference CRUD || -
Bulk import || || - Rate limiting || - Progress calc || - Nutrition calc || +-----+
+-----+ +-----+ || || +-----+ +-----+ +-----+ ||
NUTRITION SERVICE || MEAL SERVICE || AI SERVICE || || - Daily targets || - Meal logging || -
Vision proxy || || - Macro split || - Recommendation || - Meal proxy || || - BMR/TDEE calc || - History
query || - Prompt mgmt || || - Weekly summary || - Daily lifecycle || - Cache mgmt || +-----+
+-----+ +-----+ || || +-----+ +-----+ +-----+ ||
NOTIFICATION SVC || ANALYTICS SVC || SCHEDULER || || - FCM push || - Event capture || -
```

```

Cron jobs || - Email (SendGrid) || - Aggregation || - Daily gen || - In-app queue || - Metrics export ||
-Weekly gen || +-----+ +-----+
+-----+|+=====+
=====+

```

29.3 External Services

```

+=====+|
EXTERNAL SERVICES & INFRASTRUCTURE |+=====+
=====+| || +-----+ +-----+ +-----+ || |
VISION AI || MEAL AI || FCM (Firebase) || | GPT-4.1 Vision | Qwen 2.5 3B | Push notifications| || |
GPT-5 / Claude | | Gemma 3 4B | | Device token | | Food recognition | | Meal ranking | | Topic mgmt | |
+-----+ + +-----+ + +-----+ || || +-----+ + +-----+
+-----+ || | AWS S3 | | SendGrid | | CloudWatch | | | Image storage | | Email delivery | | Logs +
metrics | | | CDN origin | | Templates | | Alarms | | | Lifecycle rules | | Analytics | | Dashboards | |
+-----+ + +-----+ + +-----+
|+=====+

```

Each mobile module communicates with the backend exclusively through RESTful API endpoints exposed by the Express.js server. Inter-service communication on the backend is handled through direct function calls within the monolithic codebase during the MVP phase, with planned migration to an event-driven architecture using Redis pub/sub for the V2 release. External services are accessed through dedicated adapter classes that encapsulate API keys, retry logic, and fallback strategies. For example, the Vision AI adapter implements a circuit-breaker pattern that switches to a secondary provider (e.g., Claude Vision as fallback for GPT-4.1 Vision) if the primary provider returns more than three consecutive failures.

30. Deployment Architecture

The deployment architecture is designed for high availability, horizontal scalability, and cost efficiency on Amazon Web Services (AWS). The production environment operates within a custom Virtual Private Cloud (VPC) with two availability zones to ensure fault tolerance. All components are containerized using Docker and orchestrated via AWS Elastic Container Service (ECS) with Fargate for serverless container execution, eliminating the need for EC2 instance management.

30.1 Network Topology

The VPC is configured with an IPv4 CIDR block of 10.0.0.0/16 and contains two public subnets (for the Application Load Balancer and NAT Gateway) and two private subnets (for the ECS tasks, RDS instances, and ElastiCache nodes). An Internet Gateway provides outbound internet access for public subnet resources, while NAT Gateways in each availability zone enable outbound access from private subnet resources for API calls to external AI providers and push notification services. Security groups enforce strict traffic rules: the ALB accepts inbound HTTPS traffic on port 443 from any source and forwards it to ECS containers on port 3000. The ECS security group allows inbound traffic only from the ALB security group and allows outbound traffic to RDS (port 5432), ElastiCache (port 6379), and external HTTPS endpoints. The RDS and ElastiCache security groups accept inbound connections only from the ECS security group, creating a layered defense-in-depth network topology.

30.2 Compute Layer

The compute layer consists of two or more Express.js Docker containers running behind an Application Load Balancer (ALB). Each container is defined by a multi-stage Dockerfile that builds from a Node.js 20 Alpine base image, installs production dependencies only, and exposes port 3000. The ALB

performs health checks on the /health endpoint every 30 seconds with a threshold of 3 consecutive failures before marking a target as unhealthy. Auto-scaling policies manage the ECS service: a target tracking policy maintains an average CPU utilization of 65%, scaling out by one task when CPU exceeds 75% and scaling in when CPU drops below 50%. A minimum of 2 tasks and a maximum of 10 tasks per service are configured. Each ECS task is allocated 512 MB of memory and 0.5 vCPU for the MVP phase, with provisions to increase to 1024 MB and 1 vCPU based on production load testing results.

30.3 Data Layer and CDN

Amazon RDS for PostgreSQL is deployed in Multi-AZ configuration with a primary instance in Availability Zone A and a synchronous standby replica in Availability Zone B. The database instance class is db.t3.medium (2 vCPU, 4 GB RAM) for MVP, with automated backups retained for 30 days and point-in-time recovery enabled. Amazon ElastiCache for Redis is deployed in cluster mode with a primary node and two read replicas across availability zones, providing cache availability even during single-node failures. S3 buckets store food photographs with lifecycle rules that transition objects from Standard to Infrequent Access after 90 days and to Glacier after 365 days. Amazon CloudFront distributes S3-hosted images and the admin dashboard static assets globally, with an origin access control (OAC) policy restricting direct S3 access to CloudFront only. Amazon CloudWatch collects metrics, logs, and custom business metrics from all components. GitHub Actions CI/CD pipeline builds Docker images on every push to the main branch, runs unit and integration tests, and deploys to ECS using rolling updates with zero-downtime deployment strategy.

Component	AWS Service	Config	HA Strategy
Compute	ECS Fargate	2-10 tasks, 0.5 vCPU, 512MB	Multi-AZ, Auto-scaling
Load Balancer	ALB	HTTPS, port 443->3000	Cross-AZ routing

Component	AWS Service	Config	HA Strategy
Database	RDS PostgreSQL	db.t3.medium, Multi-AZ	Synchronous replica
Cache	ElastiCache Redis	Cluster mode, 3 nodes	Primary + 2 read replicas
Storage	S3	Standard -> IA -> Glacier	Cross-region replication (opt)
CDN	CloudFront	OAC, global edge	Automatic failover
Monitoring	CloudWatch	Logs + Metrics + Alarms	Cross-service aggregation
CI/CD	GitHub Actions	Build -> Test -> Deploy	Rolling ECS deployment

31. Security Architecture

Security is implemented as a defense-in-depth strategy with multiple overlapping layers of protection. The architecture addresses threats at the network level, application level, and data level through a combination of AWS infrastructure controls, Express.js middleware, and database-level safeguards. All security measures are designed to comply with OWASP Top 10 (2021) guidelines and are applicable to GDPR, CCPA, and HIPAA (for potential future healthcare integration) data protection requirements.

31.1 Authentication and Session Management

Authentication uses JSON Web Tokens (JWT) stored in httpOnly, secure, SameSite=Strict cookies to prevent Cross-Site Scripting (XSS) token theft and Cross-Site Request Forgery (CSRF) attacks. The access token has a 15-minute expiry and is paired with a refresh token stored in a separate httpOnly cookie with a 7-day expiry. The refresh token rotation strategy ensures that each refresh operation invalidates the previous refresh token and issues a new one, rendering stolen refresh tokens useless after a single use. Passwords are hashed using bcrypt with a cost factor of 12 rounds (approximately 250ms per

hash on modern hardware), providing strong resistance against brute-force and rainbow table attacks. Login attempts are rate-limited to 5 attempts per 15 minutes per IP address using a Redis-based sliding window counter, with progressive delays introduced after the third failed attempt.

31.2 Application-Level Security

All API endpoints use Joi schema validation to enforce strict type checking, length constraints, and format rules on every incoming request body, query parameter, and path parameter. Invalid inputs are rejected with descriptive error messages that do not leak implementation details. Cross-Origin Resource Sharing (CORS) is configured to allow requests only from the NutriAI mobile app domains and the admin dashboard origin, with all other origins receiving a 403 Forbidden response. Helmet.js is applied as global middleware to set 15 security-related HTTP headers including Content-Security-Policy, X-Frame-Options DENY, X-Content-Type-Options nosniff, and Strict-Transport-Security with a 1-year max-age. All database queries use parameterized SQL through the pg library to prevent SQL injection attacks. User-supplied strings are never interpolated into query strings. Additionally, all user-generated content (food names, notes) is sanitized using the DOMPurify library before rendering in the mobile webview or admin dashboard to prevent stored XSS attacks.

31.3 Infrastructure and Data Security

Environment variables containing API keys, database credentials, JWT secrets, and third-party tokens are encrypted at rest using AWS Secrets Manager with automatic rotation every 90 days. The application retrieves secrets at startup through the IAM role assigned to ECS tasks, eliminating the need for hardcoded credentials or .env files in the container image. All data in transit is encrypted using TLS 1.3 enforced by the ALB. The PostgreSQL RDS instance enforces SSL connections and uses the AWS-managed KMS key for encryption at rest. S3

buckets enforce server-side encryption with AES-256 (SSE-S3) and bucket policies that deny non-HTTPS access. Network-level protection includes AWS WAF (Web Application Firewall) rules on the ALB to block common attack patterns including SQL injection, XSS, and rate-based DDoS mitigation with a rate limit of 2,000 requests per 5 minutes per IP address.

32. Redis Caching Strategy

Redis serves as the primary caching layer for NutriAI, reducing database load by 70-80% for read-heavy operations and providing sub-millisecond response times for frequently accessed data. The caching strategy uses a combination of time-based expiration (TTL), write-through invalidation, and event-driven cache updates to maintain consistency between Redis and the PostgreSQL source of truth. All cached data is stored as serialized JSON strings with a standard metadata envelope containing the cached timestamp and data version.

32.1 Cache Key Design and TTL Configuration

Data Category	Key Pattern	TTL	Rationale
User Profile	profile:{userId}	1 hour	Low write frequency, high read frequency
Daily Nutrition	daily:{userId}:{date}	5 min	Changes with every meal log
Recommendations	rec:{userId}:{date}:{slot}	30 min	Regenerated after meal confirm
Food Search Results	food:search:{hash}	1 hour	Same query returns same results
JWT Session	session:{tokenId}	15 min	Matches access token expiry
Weekly Plan	weekly:{userId}:{weekStart}	24 hours	Generated once per week

Data Category	Key Pattern	TTL	Rationale
AI Response Cache	ai:{model}:{hash}	6 hours	Same prompt returns same output
Rate Limit Counter	rl:{ip}:{endpoint}	Sliding window	15 min default window

32.2 Cache Invalidation Strategies

Three complementary invalidation strategies ensure cache consistency. (1) Write-through invalidation: whenever a write operation modifies PostgreSQL data (e.g., user updates profile, meal is logged, food is edited), the corresponding Redis keys are deleted immediately within the same transaction. This ensures that the next read fetches fresh data from PostgreSQL and repopulates the cache. The deletion is performed using the DEL command for single keys or UNLINK for asynchronous deletion of multiple keys without blocking the main thread. (2) Event-driven invalidation: the backend publishes messages to Redis pub/sub channels (e.g., channel "meal:updated:{userId}") when critical mutations occur. Any ECS task subscribed to these channels proactively invalidates its local cache. (3) Time-based expiration: TTL values provide a safety net that guarantees stale data is eventually evicted even if write-through or event-driven invalidation fails due to errors or race conditions. A background garbage collection task runs every 6 hours to scan for orphaned keys (keys without corresponding TTL that may have been set without expiration by accident) and applies a default 24-hour TTL to them.

Cache warm-up occurs during two critical events: application startup and daily plan generation. At startup, the system pre-loads frequently accessed configuration data (food categories, default macro splits, supported cuisines) into Redis with a 6-hour TTL. During daily plan generation at midnight, the system populates the daily:{userId}:{date} keys for all active users in batches of 100, preventing a cache stampede when thousands of users open the app simultaneously in the morning. Cache hit/miss metrics are tracked in CloudWatch through a custom middleware that logs every cache operation with the key, hit/miss status, and latency in milliseconds.

33. AI Cost Optimization

AI cost management is critical for maintaining a sustainable unit economics model. NutriAI uses only two AI models: a commercial Vision AI for food recognition and a self-hosted Meal Recommendation AI for meal ranking. This section provides a detailed cost analysis per user per month, identifies the primary cost drivers, and describes caching and optimization strategies that keep total AI costs below \$1.00 per user per month at scale.

33.1 Vision AI Cost Breakdown

The Vision AI (GPT-4.1 Vision, GPT-5, or Claude Vision) is used exclusively for food recognition from photographs. Each image recognition call processes a single food photograph and returns a structured JSON response with identified food items, confidence scores, and estimated serving sizes. The cost per image ranges from \$0.01 to \$0.03 depending on the model selected and the image resolution. With an estimated 3 to 4 photo uploads per user per day (covering breakfast, lunch, dinner, and occasional snacks), the daily Vision AI cost per user is \$0.03 to \$0.12. Over a 30-day month, this translates to \$0.90 to \$3.60 per user per month for Vision AI alone. However, caching reduces effective calls by approximately 60%: when a user photographs a meal they have eaten before, the system matches the image hash against a Redis cache of previous recognition results. If a match is found with a confidence threshold above 0.85, the cached result is returned without invoking the Vision AI. This brings the effective Vision AI cost down to \$0.36 to \$1.44 per user per month.

33.2 Meal Recommendation AI Cost

The Meal Recommendation AI (Qwen 2.5 3B or Gemma 3 4B) is self-hosted on a GPU-equipped EC2 instance (g4dn.xlarge with 1 NVIDIA T4 GPU, 4 vCPU, 16

GB RAM at \$0.526/hour). This self-hosted model eliminates per-call API charges entirely. The cost is purely infrastructure-based: a single g4dn.xlarge instance can process approximately 200-300 recommendation requests per minute with an average latency of 150ms per call. At an average of 3 AI calls per user per day and 10,000 monthly active users, the total daily inference volume is 30,000 calls, requiring approximately 2.5 hours of GPU compute per day. Monthly infrastructure cost for the Meal AI instance is approximately \$157.80 (30 days x \$5.26/day), which amortizes to \$0.016 per user per month at 10,000 MAU. Even at 100,000 MAU, the cost scales sub-linearly to \$0.002 per user per month because a single GPU instance handles the entire inference load efficiently.

33.3 Total Cost Summary

AI Component	Cost Model	Per User/Day	Per User/Month	Notes
Vision AI	Per image (\$0.01-0.03)	\$0.01-0.04	\$0.30-1.20	After 60% cache hit reduction
Meal AI (self-hosted)	Infrastructure (g4dn.xlarge)	\$0.0005	\$0.016	10K MAU; scales sub-linearly
Total	Combined	\$0.01-0.04	\$0.32-1.22	Target: below \$1.00/user/mo

Additional optimization strategies include: (1) Batch processing of Vision AI calls during off-peak hours (11 PM to 5 AM) when cloud GPU rates may be 40-60% lower. (2) Prompt compression to reduce token count by 30% without sacrificing accuracy, achieved by using abbreviated food category names and removing verbose instructions from cached prompt templates. (3) Result deduplication across users: if multiple users upload photographs of visually similar meals (computed via perceptual hash similarity), the Vision AI result from the first user can be reused for subsequent users, with a confidence penalty applied. (4) Tiered model selection: simple, single-food images are routed to a cheaper, faster model while complex multi-food images are routed to the premium model, reducing average cost per call by an estimated 25%.

34. Error Handling

The NutriAI error handling framework provides consistent, machine-readable error responses across all API endpoints. Every error is categorized by domain, assigned a unique numeric code, and includes a human-readable message, optional detail object, and ISO 8601 timestamp. The framework supports graceful degradation: when a non-critical component fails (e.g., AI service timeout, cache miss, notification failure), the system continues to function with reduced capability rather than returning a 500 error to the user.

34.1 Error Response Format

```
{ "code": "AUTH_003", "message": "Invalid or expired refresh token", "details": { "reason": "token_expired", "expiredAt": "2025-01-15T08:30:00Z", "suggestion": "Please re-authenticate" }, "timestamp": "2025-01-15T09:00:00Z", "requestId": "req_abc123def456" }
```

34.2 Error Code Registry

Domain	Range	Examples	HTTP Status
AUTH	001-099	AUTH_001: Invalid credentials, AUTH_010: Token expired, AUTH_020: Account locked	401, 403
VALIDATION	001-099	VAL_001: Missing required field, VAL_010: Invalid email format, VAL_020: Enum out of range	400
FOOD	001-099	FOOD_001: Food not found, FOOD_010: Duplicate entry, FOOD_020: Import failed	404, 409, 422
NUTRITION	001-099	NUT_001: Target calculation error, NUT_010: Invalid macro split, NUT_020: History gap	422
AI	001-099	AI_001: Vision timeout, AI_010: Model overloaded, AI_020: Invalid response format	502, 503, 504
NOTIFICATION	001-099	NOTIF_001: FCM delivery failed, NOTIF_010: Email bounce, NOTIF_020: Unsubscribe	202 (ack)

Domain	Range	Examples	HTTP Status
SYSTEM	001-099	SYS_001: Database connection pool exhausted, SYS_010: Redis unavailable	500, 503

34.3 Graceful Degradation Strategy

When a non-critical dependency fails, the error handling middleware catches the error, logs the full context (including request ID and stack trace) to CloudWatch, and returns a degraded response rather than propagating the failure to the client. Specific degradation paths include: (1) Vision AI timeout: return an empty recognition result with code AI_001 and prompt the user to manually search for their food. The meal logging flow remains fully functional. (2) Meal AI timeout: fall back to deterministic scoring using the weighted composite algorithm (40% nutritional fit, 30% preference match, 20% variety, 10% time relevance). Users see slightly less personalized but still functional recommendations. (3) Redis cache unavailable: all operations degrade to direct PostgreSQL queries with a performance penalty of 50-200ms per request. The system logs a SYS_010 warning and automatically attempts to reconnect to Redis every 30 seconds. (4) Push notification failure: notification errors are logged with NOTIF codes and stored in a retry queue. Failed notifications are retried up to 3 times with exponential backoff (1s, 4s, 16s) before being marked as permanently failed and archived for admin review.

35. Logging

Logging infrastructure is built on Winston, the industry-standard logging library for Node.js, configured for structured JSON output with contextual enrichment. Every log entry includes a unique request ID (generated at the API gateway level and propagated through the entire request lifecycle), timestamp, log level, service context, and optional metadata fields. In production, logs are streamed to Amazon CloudWatch Logs for centralized search, analysis, and alerting.

35.1 Log Format and Levels

```
{ "timestamp": "2025-01-15T09:30:00.123Z", "level": "info", "service": "nutrition-service", "requestId": "req_abc123def456", "userId": "usr_456789", "action": "calculate_daily_targets", "message": "Daily targets calculated successfully", "metadata": { "bmr": 1650, "tdee": 2100, "targetCalories": 1800, "processingTimeMs": 12 }}
```

The logging system uses five severity levels: error (unhandled exceptions, service failures), warn (degraded states, retries, near-threshold conditions), info (successful operations, request completion), http (HTTP request/response logging with method, URL, status code, and response time), and debug (detailed variable states, query plans, cache operations, enabled only in non-production environments). In production, the http level is throttled to prevent excessive log volume from high-traffic endpoints: requests to /health, /metrics, and static asset endpoints are excluded from HTTP logging entirely.

35.2 Sensitive Data Redaction

A custom Winston format middleware intercepts every log entry before it is written and applies field-level redaction to prevent sensitive data from appearing in logs. Redacted fields include: password (any field name containing "pass"), token (any field containing "token" or "jwt"), email (partially masked as u***@domain.com), phone (masked as ***-***-1234), and any custom fields marked with a "sensitive" prefix in the metadata object. The redaction is applied recursively to nested objects and arrays. Additionally, request bodies and response bodies are never logged in full; only the URL path, query parameters (after redaction), HTTP method, status code, and response time are captured. AI call logging follows a strict policy: the system logs the AI model name, prompt hash (SHA-256 of the prompt text, not the prompt itself), response hash, processing time, token count, and cost estimate. The actual prompt text and response content are never written to logs to protect proprietary prompts and user data.

35.3 CloudWatch Integration

Winston transports are configured with two destinations in production: (1) A CloudWatch Logs transport that sends all logs (error and above) to a CloudWatch log group named `/nutriai/prod/{service-name}` with log streams automatically created per ECS task. (2) A console transport with colored output for local development and ECS task logging visible in the ECS console. CloudWatch Log Insights queries are pre-configured for common operational scenarios: "find all 5xx errors in the last hour", "show average response time by endpoint", "list all AI timeout errors with request IDs", and "count authentication failures by IP in the last 24 hours". Log retention is set to 90 days for production and 14 days for staging. A subscription filter forwards ERROR level logs to an SNS topic that triggers PagerDuty alerts for on-call engineers.

36. Monitoring

The monitoring system provides real-time observability into the health, performance, and business metrics of the NutriAI platform. Monitoring is implemented at three layers: infrastructure-level (AWS CloudWatch), application-level (custom `/health` endpoint and Prometheus exporters), and business-level (Grafana dashboards aggregating key performance indicators). Alert policies ensure that on-call engineers are notified within 60 seconds of a critical degradation.

36.1 Health Check Endpoint

```
GET /healthResponse (200 OK):{ "status": "healthy", "version": "2.1.0", "uptime": 259200, "checks": { "database": { "status": "up", "latencyMs": 3 }, "redis": { "status": "up", "latencyMs": 1 }, "s3": { "status": "up" }, "visionAI": { "status": "degraded", "latencyMs": 2500 }, "mealAI": { "status": "up", "gpuUtilization": 45 } }, "timestamp": "2025-01-15T09:30:00Z"}
```

The `/health` endpoint is called every 30 seconds by the ALB health checker and every 60 seconds by a CloudWatch Synthetics canary that performs the check

from outside the VPC. The endpoint performs a lightweight liveness check (process is running and responding on port 3000) and a readiness check (database, Redis, and S3 connectivity verified with a simple ping query). If any dependency returns a degraded status, the overall health status is set to "degraded" but the endpoint still returns HTTP 200 (since the system can function in degraded mode). A dependency failure that would prevent core functionality (e.g., PostgreSQL down) causes the endpoint to return HTTP 503, triggering an immediate ALB target deregistration and PagerDuty alert.

36.2 Performance Metrics

Metric	Source	Threshold (Warning)	Threshold (Critical)	Dashboard
Latency p50	CloudWatch	> 200ms	> 500ms	API Performance
Latency p95	CloudWatch	> 800ms	> 2000ms	API Performance
Latency p99	CloudWatch	> 2000ms	> 5000ms	API Performance
Error Rate (5xx)	CloudWatch	> 1%	> 5%	API Health
Error Rate (4xx)	CloudWatch	> 10%	> 20%	API Health
AI Success Rate	Custom metrics	< 95%	< 90%	AI Operations
AI Latency (Vision)	Custom metrics	> 3000ms	> 8000ms	AI Operations
AI Latency (Meal)	Custom metrics	> 500ms	> 2000ms	AI Operations
Cache Hit Rate	Redis INFO	< 70%	< 50%	Cache Performance
DB Connection Pool	pg pool metrics	> 80% used	> 95% used	Database Health

36.3 Grafana and Alerting

Grafana dashboards are organized into four categories: (1) API Overview, displaying request rate, error rate, latency percentiles, and active user count in

real time. (2) AI Operations, tracking Vision AI and Meal AI call volume, latency distribution, success rate, cost per 1,000 calls, and cache hit rates for AI responses. (3) Business Metrics, visualizing daily active users, meals logged per day, photo uploads per day, average daily calorie adherence, and premium conversion rate. (4) Infrastructure Health, showing ECS task count, CPU/memory utilization per task, RDS connection count and query latency, ElastiCache memory usage and eviction rate, and S3 storage growth. Alerts are configured in Grafana with PagerDuty as the notification channel. Critical alerts (p95 latency > 5s, error rate > 5%, database down) trigger a high-severity PagerDuty incident with immediate notification. Warning alerts (p95 latency > 2s, cache hit rate < 70%, AI latency elevated) trigger a low-severity PagerDuty notification routed to the team Slack channel for awareness without waking on-call engineers.

37. Notification System

The notification system delivers timely, context-aware messages to users through three channels: Push Notifications via Firebase Cloud Messaging (FCM), Email via SendGrid, and In-App Notifications stored in PostgreSQL. Each channel serves a different engagement purpose: push notifications drive re-engagement and habit formation, emails provide detailed summaries and reports, and in-app notifications surface contextual information when the user is already active.

37.1 Push Notifications (FCM)

Firebase Cloud Messaging is used for real-time push notifications on both iOS and Android. The backend maintains a `device_tokens` table that maps each user ID to one or more FCM registration tokens (supporting multi-device users). Tokens are refreshed on every app launch and invalidated when a user logs out. The FCM service adapter handles token management, message

composition, and delivery tracking. Push notifications are sent with high priority for time-sensitive alerts (meal reminders) and default priority for informational messages (weekly reports). Each notification includes a data payload with a deep link to the relevant screen in the app, a notification ID for deduplication, and the message body. The system enforces a maximum of 5 push notifications per user per day to prevent notification fatigue, with the most important notifications (meal reminders) taking priority over informational ones.

37.2 Notification Triggers

Trigger	Channel	Timing	Payload	Priority
Meal Reminder	Push	Configurable per user	{type:"meal_reminder", slot:"breakfast", deepLink:"/meal/breakfast"}	High
Water Reminder	Push	Every 2 hours (configurable)	{type:"water_reminder", currentGlasses:4, target:8}	Medium
Weekly Report	Email + Push	Sunday 8 PM	{type:"weekly_report", score:87, deepLink:"/progress"}	Medium
Achievement	Push + In-App	Real-time	{type:"achievement", badge:"7_day_streak", deepLink:"/profile"}	High
Daily Summary	In-App	After dinner	{type:"daily_summary", calories:1750, target:1800}	Low
Goal Reached	Push + Email	Real-time	{type:"goal_reached", goal:"weight", value:75}	High
Plan Ready	Push	Sunday 11:30 PM	{type:"weekly_plan_ready", deepLink:"/weekly-plan"}	Medium

37.3 Email and In-App Notifications

Email notifications are sent via SendGrid using pre-designed dynamic templates stored in the SendGrid dashboard. The Weekly Report email includes an HTML

summary of calorie adherence (percentage bar chart), macro distribution (pie chart visualization), top meals of the week, and a motivational message generated based on the user's progress. Email sending is rate-limited to 1 email per user per day to comply with SendGrid best practices and avoid spam classification. In-app notifications are stored in a notifications table in PostgreSQL with columns for `user_id`, `type`, `title`, `body`, `data` (JSONB), `read_status` (boolean), `created_at`, and `expires_at` (default 30 days). The mobile app polls for unread in-app notifications on every app launch and every 5 minutes while the app is in the foreground using a lightweight GET /notifications endpoint that returns only unread notifications. The notification service implements a unified dispatch interface that accepts a notification event, determines the appropriate channels based on user preferences, and fans out to FCM, SendGrid, and the database in parallel.

38. Analytics System

The analytics system captures, processes, and visualizes user behavior data to drive product decisions, measure feature adoption, and identify optimization opportunities. The pipeline follows an Events-First architecture: client-side events are published to Redis, consumed by an Analytics Service, persisted to PostgreSQL, and visualized in Grafana dashboards. This design separates analytics processing from the critical request path, ensuring that analytics failures never impact the core user experience.

38.1 Event Schema and Catalog

Event Name	Trigger Point	Key Properties	Volume Estimate
app_open	App foreground / cold start	userId, platform (ios/android), version, source (push/deepLink/direct)	3-5 per user/day
food_log	Meal confirmation	userId, foodId, mealSlot, source (photo/search/manual), calories	3-4 per user/day

Event Name	Trigger Point	Key Properties	Volume Estimate
photo_upload	Image captured and sent	userId, imageHash, fileSize, recognitionSuccess, processingTimeMs	2-3 per user/day
meal_selected	User taps recommendation	userId, foodId, fitScore, position (1st/2nd/3rd), mealSlot	2-3 per user/day
meal_skipped	User skips a meal card	userId, mealSlot, skippedFoodId, reason (logged_separately/auto_skip)	0-2 per user/day
profile_update	User edits profile/goals	userId, field_changed, old_value_hash, new_value_hash	0.1 per user/day
water_log	Water glass logged	userId, glasses, timestamp	4-8 per user/day
weight_log	Weight entry	userId, weight_kg, trend (up/down/same)	0.3 per user/day

38.2 Analytics Pipeline Architecture

```

+-----+ ANALYTICS PIPELINE
|+-----+ || [Mobile App] [Analytics Service] || ^ || |--
POST /analytics/event --> || {event, userId, props} || || [Backend Middleware] [Redis Stream] || | "
analytics:events" || |-- XADD analytics:events -> || {event, userId, props, || timestamp, sessionId} || || |
[Consumer Group] || || |-- Batch insert (every 10s) -----> || [PostgreSQL] analytics_events| table |
[Grafana] Dashboards+-----+

```

38.3 Key Performance Indicators

The analytics system tracks twelve core KPIs displayed on a real-time Grafana dashboard: (1) Daily Active Users (DAU) and DAU/MAU ratio as a stickiness indicator. (2) Meals Logged Per User Per Day, measuring core feature engagement. (3) Photo Log Rate, the percentage of meals logged via photo versus manual entry, indicating AI feature adoption. (4) Recommendation Acceptance Rate, the percentage of AI-recommended meals that users confirm without modification, measuring AI quality. (5) Average Daily Calorie Adherence, the percentage of users who hit their calorie target within a 10% margin. (6) 7-Day Streak Rate, the percentage of users who have logged meals for 7 consecutive days. (7) Water Logging Rate, the percentage of daily water

target achieved. (8) Weight Tracking Engagement, the percentage of users who log weight at least twice per week. (9) Session Duration, the average time users spend in the app per session. (10) Retention (D1, D7, D30), the percentage of users who return on day 1, day 7, and day 30 after installation. (11) Premium Conversion Rate, the percentage of free users who upgrade to premium within 30 days. (12) AI Cost Per User, the actual AI spend divided by MAU, tracked against the \$1.00/user/month target.

39. Admin Dashboard

The Admin Dashboard is a React-based web application accessible at `admin.nutriai.com`, serving as the central management console for platform operators, content managers, and support staff. The dashboard is authenticated separately from the mobile app using a role-based access control (RBAC) system with four roles: Super Admin (full access), Content Manager (food database and content), Support Agent (user management and tickets), and Analytics Viewer (read-only metrics). The dashboard communicates exclusively with the same Express.js backend through a dedicated `/admin/*` route prefix protected by an admin-specific JWT middleware.

39.1 Feature Modules

Module	Key Features	Access Role	Data Source
User Management	Search, view, edit, suspend, delete users; reset passwords; view login history	Super Admin, Support	PostgreSQL users table
Content Management	Create/edit/delete foods; manage categories, cuisines, diet types; bulk import via CSV	Content Manager	PostgreSQL foods table
AI Operations	View AI call logs, latency charts, cost reports; retry failed calls; clear AI cache	Super Admin	Redis + CloudWatch

Module	Key Features	Access Role	Data Source
Analytics	Real-time dashboards: DAU, retention, feature adoption, AI accuracy, revenue	All roles (read)	PostgreSQL analytics_events
System Health	ECS task status, RDS connections, Redis memory, S3 storage, ALB latency	Super Admin	CloudWatch API
Notifications	Send broadcast push notifications; schedule campaigns; view delivery reports	Super Admin, Content	FCM + PostgreSQL
Food Database	Browse 5000+ foods; search by name/cuisine/diet; bulk CSV/JSON import/export	Content Manager	PostgreSQL foods table
Support Tickets	View user reports, app crash logs, AI complaints; escalate issues	Support Agent	PostgreSQL tickets table

39.2 Food Database Bulk Operations

The Content Management module includes a bulk import tool that accepts CSV or JSON files with up to 10,000 food entries per batch. The import pipeline validates every row against a Joi schema that enforces required fields (name, serving_size, calories, protein, fat, carb), valid enum values for cuisine and diet_type, and numeric ranges for nutritional values (e.g., calories must be between 0 and 5000 per 100g). Duplicate detection uses fuzzy string matching (Levenshtein distance with a threshold of 2) against existing food names to prevent near-duplicate entries. Conflicts are flagged for manual review rather than automatically rejected. The bulk import runs asynchronously: the uploaded file is stored in S3, a background job processes it row by row, and a progress bar on the dashboard shows completion percentage. The import result summary displays total rows processed, successful inserts, duplicates flagged, validation errors (with row numbers), and a downloadable error report CSV for rows that failed validation.

The dashboard frontend is built with React 18 and the shadcn/ui component library, using Tailwind CSS for styling and Recharts for data visualization. It is deployed as a static site on CloudFront with the S3 origin serving the built artifacts. The dashboard is optimized for desktop browsers (minimum

viewport: 1280x720) and loads within 2 seconds on a 3G connection through code splitting, lazy loading of heavy chart components, and aggressive caching of static assets (Cache-Control: max-age=31536000 for hashed assets). Real-time updates for system health and analytics widgets are implemented using Server-Sent Events (SSE) from the backend, avoiding the complexity of WebSocket infrastructure for a predominantly read-only admin interface.

40. Food Knowledge Base Design

The Food Knowledge Base is the foundational data asset of NutriAI, containing nutritional profiles for over 5,000 food items spanning Indian, Mediterranean, American, East Asian, Middle Eastern, and Latin American cuisines. Each food entry is designed to serve both the deterministic filtering engine and the AI recommendation system, with rich metadata enabling precise matching and diverse meal planning. The knowledge base is stored in PostgreSQL with optimized indexes for search, filtering, and recommendation candidate retrieval.

40.1 Database Schema

```
CREATE TABLE foods ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name
VARCHAR(200) NOT NULL, name_hi VARCHAR(200), -- Hindi name (optional) cuisine
VARCHAR(50) NOT NULL, -- indian, mediterranean, american, etc. meal_type VARCHAR(20)[] NOT
NULL, -- {breakfast, lunch, dinner, snack} diet_type VARCHAR(20)[] NOT NULL, -- {veg, non-veg,
vegan, jain, eggetarian} serving_size INTEGER NOT NULL, -- in grams calories DECIMAL(6,1) NOT
NULL, -- per 100g protein DECIMAL(5,1) NOT NULL, -- per 100g fat DECIMAL(5,1) NOT NULL, --
per 100g carb DECIMAL(5,1) NOT NULL, -- per 100g fiber DECIMAL(5,1), -- per 100g sugar
DECIMAL(5,1), -- per 100g sodium DECIMAL(6,1), -- per 100g, in mg image_url TEXT, -- S3 URL for
food photo aliases JSONB DEFAULT '[]', -- ["dal fry", "yellow dal"] tags JSONB DEFAULT '[]', --
["high-protein", "low-carb", "quick"] allergens VARCHAR(50)[] DEFAULT "", -- {peanuts, dairy, gluten,
shellfish, soy} preparation_time INTEGER, -- minutes, if applicable source VARCHAR(50) NOT NULL,
-- usda, icmr, openfoodfacts, edamam, manual verified BOOLEAN DEFAULT FALSE, --
human-verified entry popularity_score DECIMAL(3,2) DEFAULT 0.50, -- 0.00-1.00, based on selection
freq created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW());
```

Critical indexes are created to support the recommendation engine's query patterns: a GIN index on the `diet_type` array for fast set-overlap queries (e.g., finding all vegetarian breakfast items), a GIN index on the `meal_type` array for meal slot filtering, a GIN index on the `allergens` array for allergen exclusion checks, a trigram index (`pg_trgm` extension) on the `name` column for fuzzy search autocomplete, and a B-tree index on (`cuisine`, `calories`) for the primary candidate retrieval query that filters by cuisine and calorie budget. The JSONB columns (`aliases` and `tags`) use GIN indexes with the `jsonb_path_ops` operator class for efficient containment queries (e.g., finding all foods tagged "high-protein"). The `popularity_score` column is updated incrementally using a decay function: each time a food is selected by a user, its score increases by 0.01, and all food scores decay by 1% weekly through a scheduled job, ensuring that trending foods surface naturally.

The `verified` boolean flag distinguishes between automatically imported entries (`verified=false`, requiring periodic human review) and entries that have been validated by a nutritionist or content manager (`verified=true`). The recommendation engine prioritizes verified foods by adding a 5-point bonus to their fit score. The `source` column tracks the data provenance of each entry, enabling the admin dashboard to filter by source and identify entries that need re-verification when a source database is updated. The `preparation_time` field enables a "quick meals" filter for users who want recommendations that can be prepared in under 15 minutes. The `name_hi` column supports Hindi-language food names for the Indian market, with plans to add additional language columns (`name_ar` for Arabic, `name_es` for Spanish) in future localization efforts.

41. Food Dataset Collection Strategy

Building a comprehensive food knowledge base requires aggregating data from multiple authoritative sources with different coverage areas, access models, and data formats. The collection strategy is designed to maximize coverage (targeting 5,000+ verified foods by MVP launch) while minimizing cost and ensuring nutritional data accuracy through cross-validation across sources. All collected data is normalized to a per-100-gram standard to enable fair comparison across serving sizes reported by different sources.

41.1 Primary Data Sources

The USDA FoodData Central (FDC) database is the primary source for American and generic food entries, providing access to over 300,000 food items with detailed nutritional profiles including calories, protein, fat, carbohydrates, fiber, sugar, sodium, vitamins, and minerals. The FDC API is free to use with a rate limit of 3600 requests per hour and requires an API key obtainable through the FoodData Central website. Data is accessed via the REST API endpoint and includes standard reference foods, branded foods, and custom food entries. The ICMR-NIN (Indian Council of Medical Research - National Institute of Nutrition) database provides authoritative nutritional data for Indian foods, which is critical for the platform's primary market. While ICMR-NIN does not offer a public API, their nutritional value tables are published as PDF documents that can be programmatically parsed using OCR and PDF extraction tools. Approximately 600 common Indian food items will be manually entered from these tables, with cross-validation against the IFCT (Indian Food Composition Tables) dataset.

Open Food Facts is a collaborative, open-source database of food products from around the world, accessible via a free REST API with a rate limit of 100 requests per hour (higher limits available for registered users). It provides bar-code-level nutritional data for packaged foods, making it invaluable for users who log branded products. Edamam offers a curated food and recipe database API with nutrition data for over 500,000 items, accessible via a developer plan at \$0.005 per call (200 free calls/day). Spoonacular provides a comprehensive food API covering ingredients, products, and recipes with nutritional data, offering a free tier of 150 requests/day and a premium tier at \$0.006 per call. Nutritionix provides a natural-language food query API and a branded food database, with pricing starting at \$0.005 per call and a free tier of 500 calls/month. Each source is assigned a priority rank for conflict resolution: ICMR-NIN ranked highest for Indian foods, USDA FDC for generic foods, and Open Food Facts for branded products. When two sources provide conflicting nutritional data for the same food item, the higher-priority source takes precedence.

41.2 Image Dataset for Vision AI

The Vision AI food recognition model requires a training dataset of labeled food images. Three primary image datasets are used: (1) Food-101, a dataset of 101,000 images across 101 food categories with 1,000 images per class, providing broad category coverage. (2) UEC FOOD-256, containing 26,684 images across 256 food categories with significantly more diverse Japanese and Asian food types, essential for the East Asian cuisine segment. (3) Recipe1M+, a dataset of over 1 million structured recipes with paired images, useful for understanding food composition and ingredient-level recognition. These datasets are used to fine-tune the Vision AI model for NutriAI's specific food categories. The fine-tuning process involves: selecting a subset of 10,000 images from these datasets that match NutriAI's 50 most common food categories, augmenting the images with rotation, brightness, and noise to improve robustness, and training a classification head on top of the pre-trained Vision AI backbone. The resulting model achieves approximately 92% top-1

accuracy on the NutriAI food categories, compared to 85% with the out-of-the-box model.

42. Data Sources

This section provides a comprehensive reference table for all data sources used in building and maintaining the NutriAI food knowledge base and AI training datasets. Each source is evaluated across eight dimensions: type of data provided, access method, rate limits, cost structure, geographic coverage, data accuracy, update frequency, and primary use case within the NutriAI ecosystem.

Source	URL	Type	Access
USDA FDC	https://fdc.nal.usda.gov/	Nutritional DB	REST API + Bulk download
ICMR-NIN	https://www.nin.res.in/	Nutrition tables	PDF (manual parse)
Open Food Facts	https://world.openfoodfacts.org/	Crowdsourced food DB	REST API (open)
Edamam	https://developer.edamam.com/	Food + Recipe DB	REST API (key)
Spoonacular	https://spoonacular.com/food-api	Food + Recipe API	REST API (key)
Nutritionix	https://www.nutritionix.com/business/api	Natural language food	REST API (key)

Source	Rate Limits	Cost	Coverage	Use Case
USDA FDC	3600 req/hr	Free	Global (US-centric)	Generic food nutritional data
ICMR-NIN	N/A (static)	Free	India	Indian food nutritional profiles
Open Food Facts	100 req/hr (unreg)	Free (CC-BY-SA)	Global (190+ countries)	Branded/package food data
Edamam	200 free/day	\$0.005/call (dev)	Global	Curated food + recipe search

Source	Rate Limits	Cost	Coverage	Use Case
Spoonacular	150 free/day	\$0.006/call (premium)	Global	Ingredient + recipe + nutrition
Nutritionix	500 free/month	\$0.005/call	US (primary)	Natural language food queries

Image Source	URL/Ref	Size	Format	Use Case
Food-101	ETH Zurich Dataset	101K images, 101 classes	JPEG, labeled	Vision AI fine-tuning (broad)
UEC FOOD-256	UEC Tokyo Dataset	26K images, 256 classes	JPEG, labeled	Vision AI (Asian cuisine)
Recipe1M+	MIT/CSAIL Dataset	1M+ recipes + images	JSON + JPEG	Vision AI (composition)

The normalization process converts all nutritional data to a per-100-gram standard regardless of the source's original serving size. This involves parsing the source's serving description (e.g., "1 cup (240ml)" or "1 piece (100g)"), extracting the numerical serving size, and dividing all nutritional values by the serving size and multiplying by 100. An automated validation step checks that normalized values fall within biologically plausible ranges (e.g., calories 0-900 per 100g, protein 0-100g per 100g, fat 0-100g per 100g). Values outside these ranges are flagged for manual review. Cross-source validation is performed for foods that appear in multiple databases: if the calorie difference between two sources exceeds 15%, the entry is flagged for human review and the higher-priority source is used as the default.

43. AI Prompt Library

The AI Prompt Library contains the exact system prompts and user message templates used for both Vision AI food recognition and Meal Recommendation AI ranking. These prompts are versioned, stored in the codebase as configuration files (not hardcoded in service files), and deployed alongside the application. Each prompt template includes metadata documenting its purpose, expected input format, expected output format, and operational constraints.

43.1 Vision AI Prompt Templates

Vision AI - System Prompt

You are NutriAI Food Recognition Assistant, a specialized food identification model. Your task is to analyze food images and identify all visible food items. CONSTRAINTS:- Identify each distinct food item visible in the image- Estimate the serving size in grams- Provide a confidence score (0.0 to 1.0) for each identification- If confidence is below 0.5, mark the item as "uncertain"- For multi-item plates, identify each item separately- Do not identify non-food items (utensils, napkins, hands)- Output MUST be valid JSON matching the response schema exactly

Vision AI - User Prompt Template

Analyze this food image and identify all food items. Context:- User cuisine preference: {cuisine_preference}- Meal type: {meal_type} (breakfast/lunch/dinner/snack)- User diet restriction: {diet_type}- User allergens: {allergens} For each identified food item, provide: 1. Name (common English name) 2. Confidence score (0.0 - 1.0) 3. Estimated serving size in grams 4. Unit description (e.g., "1 cup", "1 piece", "1 bowl")

43.2 Meal Recommendation AI Prompt Templates

Meal AI - System Prompt

You are NutriAI Meal Recommendation Engine. Your task is to rank a list of candidate meals based on how well they fit the user's remaining nutritional budget, dietary preferences, and consumption history. SCORING CRITERIA (fitScore 0-100):- Nutritional Fit (40%): How close the meal's macros match remaining targets- Preference Match (30%): Alignment with cuisine, diet type, past selections- Variety (20%): Penalty for foods eaten in the last 48 hours- Practicality (10%): Preference for quick-prep

meals during time-constrained slots

CONSTRAINTS:- Return exactly 3 recommendations (ranked by fitScore descending)- Each fitScore must be between 0 and 100- Each reason must be a concise 1-sentence explanation- Do not recommend foods the user has eaten in the last 24 hours- Do not recommend foods containing user allergens- Output MUST be valid JSON matching the response schema

Meal AI - User Prompt Template

Rank the following candidate meals for this user. User Context:- Goal: {goal} (weight_loss / muscle_gain / maintenance)- Remaining Calories: {remainingCalories} kcal- Remaining Protein: {remainingProtein}g- Remaining Fat: {remainingFat}g- Remaining Carbs: {remainingCarbs}g- Meal Slot: {mealSlot}- Time of Day: {timeOfDay}- Diet Type: {dietType}- Allergens: {allergens}- Cuisine Preference: {cuisinePreference} Recently Eaten (avoid these): {recentMeals} Candidate Meals: {candidateMeals} Return the top 3 ranked recommendations as JSON.

Prompt versioning follows a semantic versioning scheme (e.g., vision-prompt-v2.1.0, meal-prompt-v1.3.2). Each version change is tracked in a prompt_versions table in PostgreSQL that stores the version string, the full prompt text, the AI model it was designed for, the creation date, and a changelog entry explaining what changed. A/B testing of prompt versions is supported: the system can route a configurable percentage of requests to a new prompt version while the majority continue using the stable version. Success metrics (recognition accuracy, recommendation acceptance rate) are compared between versions over a 7-day evaluation period before a new version is promoted to 100% traffic.

44. Vision AI JSON Schemas

This section defines the precise JSON schemas for Vision AI request and response payloads. These schemas serve as the contract between the Backend AI Service adapter and the Vision AI model, ensuring type safety, validation, and consistent parsing of AI responses. All schemas are implemented as Joi validation objects in the backend codebase and are used to validate both outbound requests and inbound responses.

44.1 Request Schema

```
{ "$schema": "http://json-schema.org/draft-07#", "title": "VisionAIRecognitionRequest", "type": "object", "required": ["image_url", "user_id"], "properties": { "image_url": { "type": "string", "format": "uri", "description": "S3 presigned URL of the food photograph" }, "user_id": { "type": "string", "format": "uuid", "description": "Unique identifier of the user who uploaded the image" }, "context": { "type": "object", "optional": true, "properties": { "cuisine_preference": { "type": "string", "enum": ["indian", "mediterranean", "american", "east_asian", "middle_eastern", "latin_american" ] }, "meal_type": { "type": "string", "enum": ["breakfast", "lunch", "dinner", "snack" ] }, "diet_type": { "type": "string", "enum": ["veg", "non-veg", "vegan", "jain", "eggetarian" ] }, "allergens": { "type": "array", "items": { "type": "string" } } } } }
```

44.2 Response Schema

```
{ "$schema": "http://json-schema.org/draft-07#", "title": "VisionAIRecognitionResponse", "type": "object", "required": ["foods", "processingTime"], "properties": { "foods": { "type": "array", "items": { "type": "object", "required": ["foodId", "name", "confidence", "estimatedServing", "unit"], "properties": { "foodId": { "type": "string", "description": "Matched food UUID from knowledge base" }, "name": { "type": "string", "description": "Recognized food name" }, "confidence": { "type": "number", "minimum": 0, "maximum": 1, "description": "Recognition confidence 0.0-1.0" }, "estimatedServing": { "type": "number", "minimum": 1, "description": "Estimated serving size in grams" }, "unit": { "type": "string", "description": "Serving description (e.g., 1 cup, 1 bowl)" }, "calories": { "type": "number", "description": "Calories for estimated serving" }, "protein": { "type": "number", "description": "Protein for estimated serving (g)" }, "fat": { "type": "number", "description": "Fat for estimated serving (g)" }, "carb": { "type": "number", "description": "Carbs for estimated serving (g)" } } } }, "processingTime": { "type": "integer", "description": "Total processing time in milliseconds" } }
```

44.3 Error Response Schema

```
{ "$schema": "http://json-schema.org/draft-07#", "title": "VisionAIErrorResponse", "type": "object", "required": ["code", "message", "timestamp"], "properties": { "code": { "type": "string", "enum": ["AI_001", "AI_010", "AI_020", "AI_030"], "description": "AI_001=timeout, AI_010=model_overloaded, AI_020=invalid_response, AI_030=quota_exceeded" }, "message": { "type": "string", "description": "Human-readable error message" }, "details": { "type": "object", "properties": { "model": { "type": "string" }, "attempt": { "type": "integer" }, "maxRetries": { "type": "integer" } } }, "timestamp": { "type": "string", "format": "date-time" } }
```

The backend adapter wraps the raw Vision AI response in a normalized structure. If the AI returns a response that does not conform to the response schema (e.g., malformed JSON, missing required fields, confidence values outside the 0-1 range), the adapter logs an AI_020 error, attempts to parse whatever valid data it can extract, and returns a partial result with a warning flag. If no valid data can be extracted, the adapter returns the error response schema with code AI_020 and a user-friendly message prompting manual food search. All Vision AI interactions are wrapped in a circuit breaker: after 3 consecutive failures within a 60-second window, the circuit opens and all

subsequent requests are fast-failed with AI_010 (model_overloaded) for 30 seconds before a half-open state allows a single probe request to test recovery.

45. Recommendation AI JSON Schemas

This section defines the JSON schemas for the Meal Recommendation AI, covering both the request payload sent from the backend to the self-hosted Qwen/Gemma model and the structured response the model must return. These schemas are stricter than the Vision AI schemas because the Meal AI is self-hosted and must be controlled precisely to prevent hallucinated fields or malformed data from propagating to the user-facing meal cards.

45.1 Request Schema

```
{ "$schema": "http://json-schema.org/draft-07#", "title": "MealRecommendationRequest", "type": "object", "required": ["userId", "goal", "remainingCalories", "remainingProtein", "remainingFat", "remainingCarbs", "candidateMeals"], "properties": { "userId": { "type": "string", "format": "uuid" }, "goal": { "type": "string", "enum": ["weight_loss", "muscle_gain", "maintenance"] }, "remainingCalories": { "type": "integer", "minimum": 0, "maximum": 3000 }, "remainingProtein": { "type": "number", "minimum": 0 }, "remainingFat": { "type": "number", "minimum": 0 }, "remainingCarbs": { "type": "number", "minimum": 0 }, "candidateMeals": { "type": "array", "minItems": 1, "maxItems": 20, "items": { "type": "object", "required": ["foodId", "name", "calories", "protein", "fat", "carb", "cuisine", "dietType", "tags"], "properties": { "foodId": { "type": "string" }, "name": { "type": "string" }, "calories": { "type": "number" }, "protein": { "type": "number" }, "fat": { "type": "number" }, "carb": { "type": "number" }, "cuisine": { "type": "string" }, "dietType": { "type": "string" }, "tags": { "type": "array", "items": { "type": "string" } } } }, "history": { "type": "array", "description": "Foods eaten in last 48 hours [{foodId, name, timestamp}]", "items": { "type": "object", "required": ["foodId", "name", "timestamp"], "properties": { "foodId": { "type": "string" }, "name": { "type": "string" }, "timestamp": { "type": "string" } } } }, "preference": { "type": "object", "properties": { "cuisine": { "type": "string" }, "avoidCuisines": { "type": "array", "items": { "type": "string" } }, "maxPrepTime": { "type": "integer", "description": "Minutes" } } } }
```

45.2 Response Schema

```
{ "$schema": "http://json-schema.org/draft-07#", "title": "MealRecommendationResponse", "type": "object", "required": ["recommendations"], "properties": { "recommendations": { "type": "array", "minItems": 1, "maxItems": 3, "items": { "type": "object", "required": ["foodId", "name", "fitScore", "reason"], "properties": { "foodId": { "type": "string", "description": "UUID from food knowledge base" }, "name": { "type": "string", "description": "Food display name" }, "fitScore": { "type": "number", "description": "Fit score from 0 to 100" }, "reason": { "type": "string", "description": "Reason for recommendation" } } } }
```

```
minimum": 0, "maximum": 100, "description": "Composite score: 40% nutrition + 30% preference + 20%variety + 10% practicality" }, "reason": { "type": "string", "maxLength": 120, "description": "Concise1-sentence explanation for why this meal was recommended" }, "estimatedServing": { "type": "number", "description": "Suggested serving in grams" }, "calories": { "type": "number", "description": "Calories forsuggested serving" }, "protein": { "type": "number" }, "fat": { "type": "number" }, "carb": { "type": "number"} } } }, "processingTime": { "type": "integer", "description": "Inference time in ms" } }
```

The backend validates the Meal AI response using a strict Joi schema that rejects any response with fewer than 1 or more than 3 recommendations, fit scores outside the 0-100 range, missing required fields, or reason strings exceeding 120 characters. If validation fails, the system falls back to deterministic scoring, which computes the fit score using the same formula (40% nutritional fit + 30% preference + 20% variety + 10% practicality) but generates a generic reason string like "Recommended based on your nutritional goals and preferences." The deterministic fallback ensures that the user always receives recommendations even when the AI model produces unexpected output. Response caching uses a hash of the request payload (excluding userId, to enable cross-user cache sharing for similar contexts) as the Redis key, with a 6-hour TTL.

46. MVP Roadmap

The MVP roadmap spans 16 weeks divided into four phases, each delivering a functional increment of the NutriAI platform. The phased approach ensures that a usable product is available at the end of Phase 2 (week 8), with Phase 3 and Phase 4 adding AI intelligence, automation, and operational maturity. The team composition is deliberately lean: one backend engineer, one mobile engineer, and one AI/ML engineer, with the product manager and UI/UX designer operating as shared resources across phases.

46.1 Phase 1: Foundation (Weeks 1-4)

Phase 1 establishes the core infrastructure and user-facing basics. Backend milestones include: Express.js project setup with TypeScript, PostgreSQL

schema creation (users, profiles, foods, meals, daily_logs tables), JWT authentication with bcrypt password hashing, user onboarding flow (goal selection, body metrics, diet preferences, allergen configuration), nutrition calculation engine (BMI, BMR/TDEE, macro distribution), daily food logging via manual entry (text-based food search from the knowledge base), and basic dashboard API returning consumed vs. target calories and macros. The mobile team delivers: React Native project scaffolding, login/registration screens, onboarding wizard (5-step flow: goal, activity level, body metrics, diet type, allergens), manual food logging screen with search autocomplete, and a dashboard home screen showing daily calorie ring and macro bars. The food database team loads the initial 1,000 food items from USDA FDC and ICMR-NIN data with manual verification. By the end of Phase 1, a user can create an account, set goals, log food manually, and view their daily nutrition progress.

46.2 Phase 2: Vision AI Integration (Weeks 5-8)

Phase 2 introduces the photo-based food logging experience, which is the primary differentiator. The AI/ML engineer integrates the Vision AI model (GPT-4.1 Vision) with the backend, builds the image upload pipeline (mobile capture to S3 to Vision AI to food matching), and implements the confidence-based food matching logic. The mobile team builds the camera UI with food photography guidance overlay, the recognition results screen (showing identified foods with confidence scores and serving size adjustment), and the meal confirmation flow. Meal cards are introduced on the dashboard as static cards (not yet AI-powered) showing suggested meals based on time-of-day and remaining budget using deterministic filtering only. The backend team adds S3 integration with presigned URLs, the food recognition API endpoint, the meal cards API with deterministic scoring, and push notification infrastructure (FCM setup, device token management). By the end of Phase 2, the core NutriAI experience is complete: photograph a meal, see it recognized, confirm it, and view updated progress with meal card suggestions.

46.3 Phase 3: AI Intelligence (Weeks 9-12)

Phase 3 deploys the Meal Recommendation AI and adds intelligence to the platform. The AI/ML engineer sets up the self-hosted Qwen 2.5 3B model on a GPU instance, fine-tunes it on NutriAI's meal ranking training data (synthetic data generated from user preference patterns), and integrates the inference endpoint with the backend. The backend team implements the daily meal lifecycle state machine (auto-generation, post-meal recalculation, end-of-day summary), smart budget recalculation after each meal confirmation, water intake tracking and hydration reminders, weight logging with trend visualization, and the progress tracking system with weekly and monthly views. The mobile team builds the weight tracking screen, water logging widget, weekly progress chart, and meal card swipe interactions. By the end of Phase 3, recommendations are fully AI-powered, nutrition tracking is comprehensive, and the daily meal lifecycle provides a structured, automated experience.

46.4 Phase 4: Polish and Operations (Weeks 13-16)

Phase 4 adds notification automation, weekly planning, analytics infrastructure, and the admin dashboard. Push notifications are scheduled for meal reminders (configurable per user), water reminders (every 2 hours), weekly reports (Sunday evening), and achievement celebrations (streak milestones). The weekly plan generation cron job is deployed, creating seven-day templates every Sunday at 11 PM. The analytics pipeline (events to Redis to PostgreSQL to Grafana) is built with the eight core events (app_open, food_log, photo_upload, meal_selected, meal_skipped, profile_update, water_log, weight_log). The admin dashboard is deployed with user management, food database management (including bulk import), AI operation logs, and system health monitoring. End-to-end testing, performance testing (target: p95 latency under 500ms), and security auditing are conducted.

The platform is production-ready by the end of Phase 4.

46.5 Team and Timeline Summary

Phase	Weeks	Backend Focus	Mobile Focus	AI/ML Focus
1: Foundation	1-4	Auth, Onboarding, Nutrition Engine, Food DB (1000 items)	Login, Onboarding UI, Manual Logging, Dashboard	Data collection, initial 1000 foods
2: Vision AI	5-8	S3 upload, Vision AI proxy, Meal Cards API, FCM setup	Camera UI, Recognition results, Card display	Vision AI integration, food matching
3: Intelligence	9-12	Lifecycle state machine, Water, Weight, Progress	Weight/water UI, Weekly chart, Card swipe	Meal AI deploy, fine-tune, integrate
4: Operations	13-16	Notifications, Weekly plans, Analytics, Admin API	Notification settings, Weekly plan view	AI monitoring, cost optimization, tuning

47. Future Features

The NutriAI platform is designed with extensibility in mind. The modular architecture (separate services for nutrition, meals, AI, notifications, and analytics) allows new features to be added as independent modules without requiring refactoring of existing code. This section outlines the planned feature roadmap beyond the MVP, organized by strategic priority and technical complexity.

47.1 Social and Community Features

A social layer will enable users to share meal logs, progress screenshots, and achievement badges with friends and the broader NutriAI community. Features include: a public profile page showing the user's goal, current progress, and favorite meals; a community feed displaying recent meals logged by friends

with reaction capabilities (emoji reactions, comments); meal sharing where users can share their meal plan templates with friends; and group challenges (e.g., "30-Day Clean Eating Challenge" where groups of 5-20 users compete on adherence scores). The social module will use a separate social_microservice deployed behind the main ALB, with its own database tables for friendships, posts, reactions, and challenges. Privacy controls will allow users to set their profile visibility (public, friends-only, private) and control what data appears in the community feed.

47.2 Fitness Tracker and Wearable Integration

Integration with popular fitness platforms (Apple Health, Google Fit, Fitbit, Garmin Connect, Samsung Health) will enable bidirectional data exchange. Calories burned from exercise (retrieved from fitness trackers) will be automatically added to the daily calorie budget, allowing users to eat more on active days without manual adjustment. Steps data will inform the activity level component of the TDEE calculation, enabling dynamic adjustment of calorie targets based on actual daily activity rather than self-reported activity levels. Wearable integration will extend to smartwatches (Apple Watch, Wear OS) with a companion app displaying calorie rings, macro progress, and meal reminder notifications directly on the wrist. Heart rate variability (HRV) data from advanced wearables may be used to infer stress levels and recovery status, enabling the AI to suggest higher-carbohydrate meals on high-stress days and higher-protein meals on recovery days.

47.3 Restaurant Mode and Recipe Suggestions

Restaurant Mode will allow users to scan a restaurant menu (photograph or text input) and receive nutritional estimates and personalized

recommendations from the menu items. This requires building a restaurant menu database (initially populated from major chains with published nutritional data) and extending the Vision AI to handle menu text extraction (OCR) and nutritional estimation for items without published data. Recipe Suggestions will go beyond meal cards by providing full cooking instructions. When a user selects a meal card, they will have the option to view a step-by-step recipe with ingredient lists, preparation time, and instructional images. The recipe database will be initially populated from Spoonacular's recipe API and gradually enhanced with user-submitted recipes that are validated for nutritional accuracy before publication. A Grocery List feature will automatically generate a shopping list based on the user's weekly meal plan, organized by supermarket section (produce, dairy, protein, pantry, frozen), with support for multiple store formats and integration with grocery delivery APIs (Instacart, Amazon Fresh) for one-click ordering.

47.4 Voice Logging, Multi-Language, and Premium Tier

Voice Logging will leverage on-device speech-to-text (Apple Speech Framework on iOS, Google Speech API on Android) to allow hands-free food logging. Users will be able to say "I had two scrambled eggs with toast and a glass of orange juice" and the system will parse the natural language input, identify food items, estimate portions, and match them against the knowledge base. This feature requires a natural language processing (NLP) module that handles portion quantifiers ("two eggs", "a cup of rice", "a bowl of dal"), food descriptors ("scrambled", "grilled", "fried"), and compound meals ("chicken biryani", "pasta primavera"). Multi-language support will extend the app interface, food database, and AI prompts to Hindi, Arabic, Spanish, and Mandarin, targeting NutriAI's four priority geographic markets. The Premium Tier at \$9.99/month will unlock: unlimited photo recognition (free tier capped at 5/day), personalized weekly AI meal plans with grocery lists, advanced analytics with trend analysis and predictive insights, wearable integration with real-time

calorie adjustment, priority AI processing (lower latency), export data as PDF/CSV reports, and integration with dietitians/nutritionists through a telehealth-style consultation booking system. The freemium model is designed to convert 5-8% of free users to premium within the first 60 days, based on industry benchmarks for health and fitness applications.